

AugMentor 2.0: Vision–Language–Action Authoring for Egocentric Craft Video annotation

By:

**Ali Azzam
Ahmad Dia
Rafic Dergham**

Supervisor(s):

**Alina Glushkova
Juliana el Rayess**

Co-Supervisor:

Rayan El Idrissi Dafali

A report submitted in partial fulfillment of the requirements for the degree of
COMPUTER AND COMMUNICATIONS ENGINEERING

05/15/2026



Ecole Supérieure d'Ingénieurs de Beyrouth
Université Saint-Joseph de Beyrouth

Résumé

L'apprentissage d'un métier artisanal repose traditionnellement sur la présence physique d'un expert capable d'observer la technique d'un apprenti et de corriger ses erreurs en temps réel. Ce modèle de supervision est efficace mais difficile à généraliser — les experts sont rares, les séances limitées, et les apprenants qui s'entraînent seuls ne reçoivent aucun retour sur les erreurs subtiles de geste et d'outil qui déterminent la qualité de leur travail. Ce projet présente AugMentor 2.0, une plateforme web propulsée par l'intelligence artificielle qui comble ce manque en analysant automatiquement la vidéo d'entraînement d'un apprenant, en la comparant à une référence enregistrée par un expert, et en fournissant un retour structuré et personnalisé — sans nécessiter la présence de l'expert.

La plateforme traite la vidéo égocentrique de l'apprenant à travers une séquence d'étapes de vision par ordinateur et de traitement du signal, qui extraient la position et l'orientation de l'outil de coupe image par image, comparent le tracé de découpe et l'angle de la lame de l'apprenant à la référence experte, et détectent les tremblements involontaires de la main. Chaque étape applique des critères explicites et mesurables pour déclarer une erreur — garantissant que les évaluations du système sont objectives, reproductibles et entièrement explicables. Les résultats sont présentés à l'apprenant via une interface ludifiée qui l'invite à identifier ses propres erreurs avant de révéler l'évaluation du système, et un modèle de langage formé à communiquer les conclusions dans un langage clair et encourageant, sans jargon technique.

Le système a été validé à travers une série de tests contrôlés. Le modèle de détection d'outil a atteint une précision quasi parfaite sur le jeu de données spécifique au domaine, sans aucune détection manquée ni fausse alarme. Le module de détection des vibrations a correctement identifié un tremblement de main dans une vidéo de test et a correctement retourné une absence de détection sur une vidéo à mouvement stable. Une évaluation de bout en bout sur un test de contrainte délibéré — où un apprenant a effectué un mouvement de coupe en zigzag exagéré face à une référence experte en ligne droite — a détecté quatre erreurs distinctes, localisé chacune correctement dans le temps et dans l'espace, et généré des instructions de correction spécifiques pour chacune via le modèle de langage. L'évaluation complète s'exécute en moins de deux minutes sur un ordinateur portable grand public équipé d'une carte graphique.

Les résultats démontrent que l'évaluation fine des compétences artisanales à partir de vidéo est réalisable avec les outils d'intelligence artificielle actuels, sans nécessiter de grands ensembles de données d'entraînement ni l'intervention d'un expert au moment de l'évaluation. AugMentor 2.0 fournit une base concrète et extensible pour l'éducation professionnelle assistée par l'IA, avec le potentiel de rendre un retour de niveau expert accessible aux apprenants quelle que soit leur localisation ou leur accès à une supervision en personne.

Abstract

Learning a craft skill traditionally requires the physical presence of an expert who can observe a learner's technique and correct their mistakes in real time. This model of supervision is effective but difficult to scale — experts are scarce, sessions are limited, and learners who practice alone receive no feedback on the subtle hand and tool errors that determine the quality of their work. This project presents AugMentor 2.0, an AI-powered web platform that bridges this gap by automatically analysing a learner's craft practice video, comparing it against a registered expert reference, and delivering structured, personalised feedback — without requiring the expert to be present.

The platform processes the learner's egocentric video through a sequence of computer vision and signal processing stages that extract the position and orientation of the cutting tool frame by frame, compare the learner's cutting path and blade angle against the expert's reference, and detect involuntary hand tremor. Each stage applies explicit, measurable criteria to declare an error — ensuring that the system's evaluations are objective, reproducible, and fully explainable. The results are presented to the learner through a gamified interface that challenges them to identify their own mistakes before revealing the system's assessment, and a language model trained to communicate the findings in plain, encouraging language without technical jargon.

The system was validated through a series of controlled tests. The tool detection model achieved near-perfect accuracy on the domain-specific dataset, with zero missed detections and zero false alarms. The vibration detection module correctly identified hand tremor in a test video and correctly returned no detection on a steady-motion video. An end-to-end evaluation on a deliberate stress test — where a learner performed an exaggerated zigzag cutting motion against a straight-line expert reference — detected four distinct errors, correctly localised each one in time and space, and generated specific correction instructions for each through the language model layer. The complete evaluation runs in under two minutes on a standard gaming laptop.

The results demonstrate that fine-grained craft skill assessment from video is achievable with current AI tools, without requiring large training datasets or expert involvement at evaluation time. AugMentor 2.0 provides a concrete and extensible foundation for AI-assisted vocational education, with the potential to make expert-level craft feedback accessible to learners regardless of their location or access to in-person supervision.

Acknowledgements

We would like to express our sincere gratitude to our supervisors, Professor Alina Glushkova and Professor Juliana El Rayess, for their guidance, availability, and constructive feedback throughout the duration of this project. Their direction helped shape both the technical approach and the academic rigour of this work.

We are equally grateful to our co-supervisor, Rayan El Idrissi Dafali, whose technical insight and hands-on support during the implementation phases were invaluable. This work would not have reached its current level of completeness without his involvement.

We would like to thank the Ecole Supérieure d'Ingénieurs de Beyrouth and Université Saint-Joseph de Beyrouth for providing the academic framework and resources that made this project possible, and Mines Paris for the expert video dataset that served as the reference foundation for the evaluation pipeline.

We are also grateful to our families and friends for their patience and encouragement throughout a demanding final year.

Contents

Résumé	2
Abstract.....	3
List of figures	6
List of tables.....	7
Acronyms.....	8
1 Introduction	9
2 Background, Research and Related Work	10
2.1 Historical Evolution: From Symbolic AI to Multimodal Learning	10
2.2 State of the Art in Motion Analysis and Visual Understanding.....	10
2.3 Critical Evaluation of Existing Approaches	13
2.4 Positioning and Contribution of the Proposed Work	14
3 Specifications.....	15
3.1 Functional Requirements	15
3.2 Non-Functional Requirements	18
3.3 Conceptual Model	23
4 Solution Design.....	25
4.1 Design Alternatives and Comparison.....	25
4.2 Solution Modeling.....	30
5 Implementation, Test, Evaluation and Iterative Improvements	36
5.1 Technical Environment and Development Setup.....	36
5.2 Development Methodology	39
5.3 Phase 1 — Model Exploration and Benchmarking	40
5.4 Phase 2 — Pipeline Implementation.....	56
5.5 Frontend Implementation	64
5.6 Implementation Challenges.....	66
5.7 Test and Evaluation	68
6 Impact of the Proposed Solution	74
7 Conclusion	76
8 References	78
Appendix 1	80
Appendix 2 - Declaration of AI Usage	87

List of figures

Figure 1 - Gantt chart	10
Figure 2 - Conceptual Model	23
Figure 3 - System Context Diagram	30
Figure 4 - High-Level Architecture Diagram	31
Figure 5 - Component Diagram	31
Figure 6 - Sequence Diagram	32
Figure 7 - VLM Prompt Construction Diagram	33
Figure 8 - Activity Diagram	34
Figure 9 - AugMentor 2.0 application stack	38
Figure 10 - Training batch sample	41
Figure 11 - Validation batch sample	41
Figure 12 - Dataset label distribution	42
Figure 13 - Training and validation loss curves and metric progression across epochs	42
Figure 14 - F1-confidence curve	43
Figure 15 - Precision-confidence curve	43
Figure 16 — Recall-confidence curve	44
Figure 17 - Precision-recall curve	44
Figure 18 - Normalised confusion matrix	44
Figure 19 - SAM2 segmentation mask	46
Figure 20 - SAM2 tracking quality	46
Figure 21 - Blade angle estimation	48
Figure 22 - DTW-aligned angle comparison	48
Figure 23 - vibration detection comparison	50
Figure 24 - RAFT optical flow	51
Figure 25 - V-JEPA 2.1 segment-level semantic similarity	52
Figure 26 - Fast-HaMeR 3D	52
Figure 27 - VLM-generated explanation output rendered in the AugMentor 2.0	54
Figure 28 - AugMentor 2.0 evaluation pipeline	56
Figure 29 - Aligned corridor preview	58
Figure 30 - Angle error preview	59
Figure 31 - Trajectory error preview	59
Figure 32 - Gamified error marking interface	60
Figure 33 - Score result	60
Figure 34 - SAM2 CPU-only usage	62
Figure 35 - SAM2 GPU memory and utilisation profile on RTX 3060	62
Figure 36 - SAM2 GPU memory and utilisation profile on GTX 1650	63
Figure 37 - MediaPipe CPU usage	63
Figure 38 - Terminal output showing YOLO pre-pass timing	64
Figure 39 - Terminal output showing SAM2 pipeline timing	64
Figure 40 - Home interface	65
Figure 41 - Courses interface	65
Figure 42 - Course content and chapters	66
Figure 43 - Compare Studio	66

List of tables

Table 1 - <i>Comparison of existing approaches</i>	14
Table 2 - Summary of Functional Requirements	18
Table 3 - Summary of Non Functional Requirements	21
Table 4 - Quantitative Decision Matrix	26
Table 5 - Low-Level Component Selection	30
Table 6 - Software Resources	35
Table 7 - Hardware Resources	36
Table 8 - Dataset and Infrastructure Resources	36
Table 9 - Hardware configurations used during development	37
Table 10 - Key Python dependencies and roles	37
Table 11 - YOLOv8s validation results	45
Table 12 - SAM2 tracking quality metrics	47
Table 13 - Vibration detection results	50
Table 14 - Two-run VLM output comparison on identical input	53
Table 15 - Comparative assessment of all evaluated models	55
Table 16 - Phase 1 runtime comparison	61
Table 17 - Per-stage runtime breakdown for a complete learner evaluation run on RTX 3060	63
Table 18 - Detected error events	69
Table 19 - DTW angle summary metrics	70
Table 20 - Requirements Satisfaction Summary	73

Acronyms

AI	–	Artificial Intelligence
ML	–	Machine Learning
CNN	–	Convolutional Neural Network
VLM	–	Vision–Language Model
CV	–	Computer Vision
DTW	–	Dynamic Time Warping
API	–	Application Programming Interface
REST	–	Representational State Transfer
HTTP	–	Hypertext Transfer Protocol
JSON	–	JavaScript Object Notation
UI	–	User Interface
UX	–	User Experience
SPA	–	Single Page Application
GPU	–	Graphics Processing Unit
CUDA	–	Compute Unified Device Architecture
FPS	–	Frames Per Second
DB	–	Database
SQL	–	Structured Query Language
SRS	–	Software Requirements Specification
NFR	–	Non-Functional Requirement
KPI	–	Key Performance Indicator
GDPR	–	General Data Protection Regulation
ISO	–	International Organization for Standardization
CI/CD	–	Continuous Integration / Continuous Deployment
UML	–	Unified Modeling Language
ROI	–	Region of Interest
YOLO	–	You Only Look Once (Object Detection Model)
LMS	–	Learning Management System
SDK	–	Software Development Kit
RAM	–	Random Access Memory
OS	–	Operating System
SAM	–	Segment Anything Model
V-JEPA	–	Video Joint Embedding Predictive Architecture
HaMeR	–	Hand Mesh Recovery

1 Introduction

The rapid progress of Artificial Intelligence (AI) and Computer Vision (CV) has significantly transformed the way human motion and technical skills are analyzed. Intelligent systems are increasingly deployed in domains such as sports analytics, industrial inspection, and surgical training to extract structured information from video and generate meaningful performance feedback. However, vocational education in artisan crafts remains largely dependent on traditional apprenticeship models, where learning occurs through observation and in-person correction by expert craftsmen. While effective, this model is time-intensive, difficult to scale, and lacks objective, data-driven evaluation mechanisms.

Artisan activities — including woodworking, pottery, leather crafting, and fine cutting tasks — require precise hand coordination, controlled tool manipulation, stable tool angles, and smooth motion trajectories. These movements are often subtle and difficult to describe explicitly, making it hard for learners to identify what they are doing wrong without an expert present. The core question this project addresses is straightforward: given a video of an expert performing a craft task and a video of a learner attempting the same task, can a system automatically identify where and how the learner's performance deviates from the expert's, and explain those differences in a clear and actionable way?

AugMentor 2.0 is an AI-powered web-based platform built around this idea. The system compares expert and learner videos by tracking the tool across frames and computing differences in trajectory, tool angle, vibration, and hand positioning. These differences are localized as specific error events in the learner's video. The learner can then watch their video, click on the errors they notice, and receive an AI-generated explanation of what went wrong, why it matters, and what to do differently — all grounded in the measurements the system computed.

AugMentor 2.0 builds upon an initial proof-of-concept, AugMentor 1.0, which demonstrated the feasibility of extracting basic motion representations and generating simple graphical overlays. The present work introduces a full expert-to-learner comparison pipeline, precise error localization, richer video annotations, a gamified scoring system, and an AI explanation layer strictly guided by computed results.

The intended beneficiaries are artisan craft learners who lack continuous access to expert instruction, training institutions seeking scalable evaluation tools, and researchers working at the intersection of computer vision and skill assessment. The system is particularly relevant in remote learning contexts where feedback without a physically present instructor is essential.

To achieve these objectives, the project adopted a hybrid engineering approach combining systematic model exploration with iterative pipeline development. An initial research phase evaluated multiple perception models under the specific constraints of craft video analysis, where standard off-the-shelf approaches showed significant limitations. This experimental phase directly informed all design decisions, leading to a modular pipeline that detects and tracks the tool, measures deviations in trajectory and angle against an expert reference, detects vibration, and estimates hand-to-tool distance. Detected errors are surfaced in an interactive video interface where the learner engages with their own performance, and an AI language model generates targeted explanations for each identified error.

The system was validated on a straight-line cutting task, demonstrating reliable error detection, gamified learner interaction, annotated video output, and structured feedback grounded in computed measurements.

The project was conducted between January 19 and May 15, 2026, following the development phases illustrated in Figure 1. The remainder of this report is organized as follows. Section 2 reviews related work on motion analysis and skill evaluation. Section 3 specifies the system requirements. Section 4 presents the solution design and key decisions. Section 5 details the implementation and evaluation results. Section 6 assesses the broader impact, and Section 7 concludes with observations on achievements, limitations, and future directions.

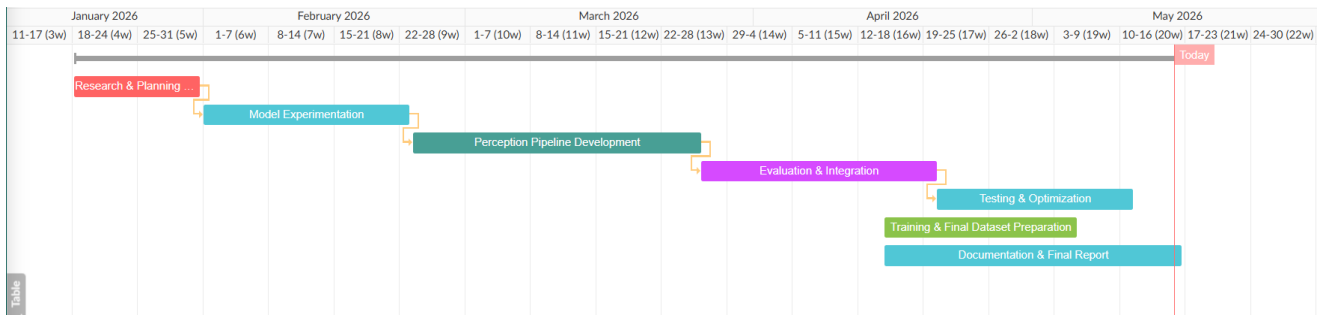


Figure 1 - Gantt chart

2 Background, Research and Related Work

2.1 Historical Evolution: From Symbolic AI to Multimodal Learning

The development of AI for video understanding has passed through several distinct phases, each expanding what machines can extract from visual data. Early systems relied on symbolic reasoning and hand-crafted rules, which worked in controlled environments but broke down quickly when faced with the complexity and variability of real-world video. The introduction of machine learning allowed data-driven modeling, though these methods still struggled with high-dimensional visual inputs.

The major shift came in the 2010s with deep learning and Convolutional Neural Networks, which enabled effective feature extraction from images and video at scale [5]. This gave rise to practical pose estimation and object detection systems — tools that could identify where a hand is, where a tool is, and how a body is positioned, frame by frame. These systems were transformative but remained fundamentally observational: they could detect what was happening, not evaluate how well.

More recent developments have pushed toward video understanding at a higher level of abstraction. Self-supervised models such as V-JEPA [18] and V-JEPA 2.1 [19] learn motion representations without labeled data, capturing temporal patterns across entire sequences. Segmentation models such as SAM 2 [17] extend tracking robustness under occlusion and complex backgrounds. Vision–Language Models add the ability to generate natural language descriptions of visual content. The field has clearly moved from detection toward understanding — yet, as discussed throughout this section, these advances remain largely disconnected from one another, with few systems integrating them into a unified evaluation framework.

2.2 State of the Art in Motion Analysis and Visual Understanding

Approaches relevant to fine-grained skill evaluation can be grouped into four complementary layers: perception-based methods that extract structured data from raw video; motion representation and temporal comparison techniques that measure how sequences relate to one another; representation learning models that capture high-level patterns; and multimodal systems that combine visual and language understanding. Each layer contributes something important, but each also carries limitations that make it insufficient on its own.

2.2.1 Perception-Based Methods

Pose estimation frameworks such as MediaPipe and OpenPose transform raw video frames into geometric representations — typically sets of keypoints representing joints or hand landmarks. MediaPipe, for instance, provides 21 keypoints per hand, enabling the computation of angles, trajectories, and velocities directly from these positions. Object detection models such as YOLO complement this by identifying and localizing objects in the scene, including tools and materials relevant to the task being performed.

These methods are foundational because they convert unstructured visual data into something measurable. Without this layer, no quantitative evaluation is possible. However, their limitations are significant. They detect what is present in a frame rather than evaluating how well something is being done. Their performance degrades in conditions common to craft video: occlusion, non-standard viewpoints, and domain-specific tools not well represented in their training data. Perhaps most importantly, they produce no comparison mechanism — knowing where a hand or tool is tells you nothing about whether that position is correct relative to an expert reference.

As a result, perception-based methods are a necessary starting point, but insufficient by themselves for skill assessment.

2.2.2 Motion Representation and Temporal Comparison

Once structured data is extracted, the next challenge is comparing it meaningfully. Motion representation techniques describe how movement evolves over time through trajectories, velocity profiles, and temporal patterns. These representations bridge the gap between discrete frame-level detections and continuous motion sequences that can be analyzed holistically.

The central difficulty in comparing expert and learner performances is that they are rarely synchronized — a learner may perform the same action more slowly, pause at different moments, or accelerate through sections the expert handles carefully. Dynamic Time Warping (DTW) addresses this directly by finding an optimal alignment between two sequences, allowing comparison even when they differ in speed or timing [13]. This makes DTW particularly valuable for expert–learner comparison scenarios, where the goal is to identify deviations in execution rather than differences in pace. Related work such as FinePseudo [14] further demonstrates how temporal alignment can be leveraged for fine-grained action comparison, reinforcing the relevance of DTW-based approaches in skill evaluation contexts.

These techniques provide a solid quantitative basis for detecting where performances diverge. Their limitation, however, is interpretive: DTW can tell you that two sequences differ at a given point, but not why that difference matters or what the learner should do about it. The output is numerical, not explanatory. This gap between measurement and meaning is one of the central challenges that AugMentor 2.0 addresses.

2.2.3 Optical Flow and Motion Analysis

Optical flow methods estimate the apparent motion of pixels between consecutive video frames, producing dense vector fields that describe how each region of the image is moving. Rather than relying on detected keypoints or explicit object localization, optical flow operates directly on pixel intensity patterns, making it applicable in conditions where landmark detection may fail. RAFT (Recurrent All-Pairs Field Transforms) [21] represents a significant advance in this area, achieving state-of-the-art accuracy in optical flow estimation through a recurrent iterative update mechanism that refines flow predictions across multiple steps.

In the context of craft skill analysis, optical flow offers a useful complementary signal. While trajectory and angle metrics capture global tool movement, optical flow captures local motion patterns at the pixel level — making it particularly suited for detecting subtle, high-frequency movements such as tool vibration or instability that would be invisible to landmark-based approaches. However, optical flow is sensitive to lighting changes, camera motion, and background clutter, which can introduce noise in egocentric video settings. It also does not inherently provide semantic information about which region of the frame is relevant, requiring additional processing to isolate tool-related motion from irrelevant background movement.

2.2.4 Representation Learning and Video Understanding

Recent advances in deep learning have introduced representation learning approaches that capture high-level patterns directly from video data without relying on explicit geometric features. Models such as V-JEPA [18] and its improved version V-JEPA 2.1 [19] learn by predicting missing or future segments of a video sequence, forcing the model to internalize the temporal structure of motion. The result is an embedding space where videos with similar motion patterns cluster together, enabling comparison at a semantic level.

These models are particularly useful for capturing holistic, contextual information — the kind that is difficult to express through keypoints alone. They can recognize that two performances are structurally similar even if their exact trajectories differ, or identify which segment of a learner's video diverges most from the expert. This makes them valuable as a high-level screening tool.

The limitation is precision. Representation learning models operate at a level of abstraction that makes their outputs — similarity scores, embeddings — difficult to interpret directly. They cannot produce the specific, measurable values that a learner needs to improve: how many degrees the tool was tilted, how far the trajectory deviated, at exactly which moment. For that, structured geometric methods are still required.

2.2.5 Multimodal and Advanced Perception Systems

The most advanced current approaches combine multiple sources of information to overcome the limitations of any single method. Segmentation models such as SAM 2 [17] extend perception beyond keypoints by tracking entire regions of interest across frames, making them considerably more robust to occlusion and viewpoint variation than landmark-based approaches. Three-dimensional reconstruction models such as HaMeR [16] and Fast-HaMeR [20] provide detailed geometric hand representations that capture fine-grained articulation beyond what 2D landmarks can express, though at a significant computational cost.

At a higher level, Vision–Language Models integrate visual understanding with natural language generation, enabling systems to produce human-readable descriptions and explanations of visual content [11]. This capability is particularly relevant for educational applications, where the goal is not just to detect an error but to communicate it in a way the learner can act on. Work on hybrid adaptivity in education [4] further highlights the value of combining AI-generated feedback with structured human interaction, reinforcing the pedagogical direction taken in this project.

Despite these strengths, each approach remains specialized. Segmentation models handle spatial tracking but do not evaluate motion quality. Reconstruction models provide geometry but are computationally intensive. Vision–Language Models generate explanations but, without structured input, risk producing outputs that are inconsistent or disconnected from actual measurements — a grounding problem that requires careful system design to address.

2.3 Critical Evaluation of Existing Approaches

Examining the literature as a whole, a clear structural gap emerges. Three functional layers exist largely in isolation: perception systems that detect geometry and context; generative systems that describe what they observe; and comparison systems that measure similarity between sequences. What is missing is integration — a framework that connects these layers into a coherent pipeline for skill evaluation.

Most research answers the question of what is happening. Skill assessment requires a fundamentally different question: how well is it being done, relative to a reference? Some domain-specific systems attempt to bridge this gap. Surgment, for example, combines segmentation with semantic interpretation for surgical video analysis, enabling region-level querying and structured evaluation. It demonstrates that fine-grained evaluation is achievable — but within a controlled domain with predefined workflows, and without generalizing to open settings like artisan craft learning where tools, viewpoints, and execution styles vary considerably.

A further practical limitation is the absence of publicly available expert–learner paired datasets for craft domains. This makes purely data-driven evaluation approaches difficult to apply, and reinforces the need for systems that can function with a single expert reference rather than large labeled collections.

The table below summarizes the main approaches reviewed and their respective strengths and limitations.

Table 1 - Comparison of existing approaches

Method	Main Use	Strengths	Limitations
MediaPipe OpenPose	Hand and pose keypoint extraction	Lightweight, real-time, interpretable	Sensitive to occlusion and egocentric viewpoints
YOLO	Object and tool detection	Fast, effective for general objects	Weak on domain-specific tools without fine-tuning
DTW	Temporal sequence alignment	Handles speed variation, enables expert–learner comparison	No semantic interpretation
Optical Flow RAFT	Dense pixel motion estimation	Captures subtle motion and vibration	Sensitive to lighting and background noise
V-JEPA 2.1	Video representation learning	Captures high-level motion patterns	Not directly interpretable, lacks precise geometric metrics
SAM 2	Video segmentation and tracking	Robust to occlusion, tracks full regions	Does not evaluate motion quality
Fast-HaMeR	3D hand reconstruction	Detailed hand geometry	Computationally intensive
Vision–Language Models	Explanation generation	Human-readable feedback	May hallucinate without grounded structured input
Surgment	Surgical skill assessment	Fine-grained analysis in controlled settings	Domain-specific, not generalizable

2.4 Positioning and Contribution of the Proposed Work

AugMentor 2.0 is positioned at the intersection of the layers described above, with the explicit goal of integrating them into a unified expert-to-learner comparison pipeline. Rather than relying on any single approach, the system combines domain-specific object detection for tool localization, segmentation-based tracking for trajectory extraction, geometric analysis for angle and deviation measurement, optical flow for vibration detection, hand landmark detection for distance estimation, and a controlled Vision–Language Model layer for explanation generation — where the model is strictly constrained by computed measurements rather than left to interpret video freely.

The contribution of this work lies not in individual components, which draw on established models and techniques, but in their integration into a coherent evaluation framework designed for the specific challenges of egocentric craft video. The system requires only a single expert reference video, making it practical in domains where large labeled datasets do not exist — addressing one of the key limitations identified across the reviewed literature.

3 Specifications

This section outlines the intended functionality, system constraints, and performance expectations of AugMentor 2.0. Requirements are defined in accordance with the ISO/IEC/IEEE 29148:2011 standard, which provides provisions for the engineering of requirements for systems and software products throughout the life cycle. It defines the construct of a good requirement, provides attributes and characteristics of requirements, and discusses the iterative and recursive application of requirements processes. All requirements are phrased as tangible functions that users and the system can see and act upon.

3.1 Functional Requirements

FR-1: User Authentication and Access Control

The system shall allow users to:

- Create an account
- Authenticate using valid credentials,
- Log out securely.

The system shall restrict access to evaluation features to authenticated users only.

Verification: Register/login test with valid and invalid credentials.

FR-2: Course and Expert Content Access

The system shall provide users with access to available artisan courses and structured expert video chapters.

Users shall be able to:

- Browse available courses
- Select a chapter,
- View the associated expert reference video.

Verification: User navigation test — browse courses and access chapter video.

FR-3: Practice Video Submission

The system shall allow authenticated users to upload a practice video associated with a selected chapter for evaluation. The system shall validate the upload prior to processing. Format and duration constraints are defined under Non-Functional Requirements.

Verification: Upload test — valid video accepted, invalid rejected.

FR-4: Domain Validation and Rejection

The system shall automatically analyze each uploaded practice video to verify its relevance to the selected artisan domain. If the uploaded video does not correspond to the expected craft context, the system shall:

- Reject the video,
- Notify the user with an appropriate message,
- Prevent further evaluation processing.

Verification: Upload mixed dataset of valid and unrelated videos and verify correct rejection behavior.

FR-5: Structured Motion and Tool Extraction

The system shall extract structured motion information from both expert and learner videos, including:

- Tool detection and localization across video frames,
- Blade tip trajectory extraction through segmentation-based tracking,
- Tool orientation angle estimation,
- Hand landmark detection for distance computation between the non-tool hand and the tool.

This extraction shall provide the structured data required for subsequent evaluation.

Verification: System generates tool detections, trajectory data, angle estimates, and hand landmark coordinates from input video.

FR-6: Motion Alignment and Quantitative Evaluation

The system shall:

1. Align the learner's motion sequence with the expert reference sequence,
2. Compute quantitative deviation metrics,
3. Generate a normalized performance score based on deterministic evaluation logic.

Deviation metrics shall include, but are not limited to :

- Tool angle deviation relative to the expert reference,
- Trajectory deviation against the expert corridor,
- Tool vibration intensity derived from optical flow analysis,
- Spatial distance between the non-tool hand and the tool.

Verification: Identical input produces consistent deviation metrics and score.

FR-7: Feedback Generation

The system shall generate evaluation feedback in two complementary forms:

- Visual overlays in an annotated video highlighting motion deviations,
- Structured textual feedback grounded in computed quantitative metrics.

Textual feedback shall reference measurable results and shall not include unsupported claims.

Verification: Visual overlays displayed and textual feedback references computed metrics.

FR-8: Evaluation History Management

The system shall store evaluation results for each user and allow retrieval of previous evaluation attempts. Stored results shall include:

- Computed deviation metrics,
- Performance score,
- Associated chapter reference.

Verification: User retrieves stored evaluation results from database.

FR-9: Gamification and Interactive Progress Tracking

The system shall provide gamified features to support user engagement, including:

- Performance score visualization,
- Chapter completion indicators,
- Comparison with previous attempts,
- Interactive error identification, where users can mark perceived errors directly on the evaluation video and receive a score reflecting the accuracy of their identification.

Verification: Verify progress visualization, user interaction through error marking, and scoring accuracy.

Table 2 - Summary of Functional Requirements

FR ID	Requirement Name	Priority	Verification Method
FR-1	User Authentication and Access Control	Must	Register/login test with valid and invalid credentials
FR-2	Course and Expert Content Access	Must	User navigation test: browse courses and access chapter video
FR-3	Practice Video Submission	Must	Upload test: valid video accepted, invalid rejected
FR-4	Domain Validation and Rejection	Must	Upload mixed dataset and verify correct rejection behavior
FR-5	Structured Motion and Tool Extraction	Must	System generates tool detections, trajectory, angle, and landmark data
FR-6	Motion Alignment and Quantitative Evaluation	Must	Identical input produces consistent deviation metrics and score
FR-7	Feedback Generation	Must	Visual overlays displayed and textual feedback references computed metrics
FR-8	Evaluation History Management	Should	User retrieves stored evaluation results from database
FR-9	Gamification and Interactive Progress Tracking	Should	Verify progress visualization, error marking, and scoring accuracy

3.2 Non-Functional Requirements

3.2.1 Quality attributes and technical constraints

3.2.1.1 Performance and Computational Efficiency

The system shall operate within defined computational limits to ensure timely evaluation and responsive user interaction.

NFR-P1: Processing Latency

The system shall process practice videos of maximum duration 30 seconds within an acceptable latency under the defined hardware configuration.

Verification: Benchmark testing under controlled hardware conditions.

NFR-P2: Metric Reproducibility

Repeated evaluation of identical input under identical system configuration shall produce deviation metrics within $\leq 5\%$ variation.

Verification: Repeated execution comparison tests.

Standards, laws and regulations

All standards, laws and regulations related to the project's domain or any sub-domain should be identified, described and referenced. For example: HIPAA for healthcare, GDPR for data protection, etc...

- Ethical: privacy issues, anonymity, transparency of the procedures.
- Health and Safety: safety standards, impact of the product on the user.
- Political: need for government approval, ...

3.2.1.2 Reliability and Robustness

The system shall maintain stable operation under expected and degraded input conditions.

NFR-R1: Fault Tolerance

The system shall handle corrupted, unsupported, or incomplete video uploads without system failure and shall return an informative error message.

Verification: Controlled invalid input testing.

NFR-R2: Input Format and Duration Constraints

The system shall enforce the following constraints on uploaded practice videos:

- Supported formats: MP4 and MOV,
- Maximum duration: configurable, default ≤ 30 seconds.

Videos violating these constraints shall be rejected prior to processing.

Verification: Upload validation testing.

3.2.1.3 Security and Data Protection

The system shall ensure secure communication and protection of stored data.

NFR-S1: Secure Communication

All client-server communication shall use HTTPS with TLS encryption.

Verification: SSL inspection and security configuration validation.

NFR-S2: Access-Controlled Storage

User-uploaded videos and evaluation results shall be stored in authenticated, access-controlled storage.

Verification: Unauthorized access attempt testing

3.2.1.4 Architectural Constraints and Maintainability**NFR-A1: Layered Architecture**

The system shall maintain separation between the User Interface Layer, Validation Module, Evaluation Engine, Vision–Language Module, and Data Storage Layer.

Verification: Architectural inspection.

NFR-A2: Deterministic Evaluation Logic

Performance scoring shall be derived strictly from defined mathematical formulas without stochastic components.

Verification: Code inspection and repeated execution validation.

3.2.1.5 Explainability and Transparency**NFR-E1: Grounded Textual Feedback**

All generated textual explanations shall reference computed quantitative deviation metrics and shall not include unsupported or unverifiable claims.

Verification: Manual audit of generated outputs

Table 3 - Summary of Non Functional Requirements

NFR ID	Category	Requirement Name	Verification Method
NFR-P1	Performance	Processing Latency	Benchmark test using 30-second videos under defined hardware
NFR-P2	Performance	Metric Reproducibility	Repeated evaluation test: metric variation $\leq$ 5%
NFR-R1	Reliability	Fault Tolerance	Upload corrupted/invalid file and verify controlled rejection
NFR-R2	Reliability	Input Format and Duration Constraints	Upload unsupported format or excessive duration and verify rejection
NFR-S1	Security	Secure Communication	Validate HTTPS/TLS configuration
NFR-S2	Security	Access-Controlled Storage	Attempt unauthorized data access and verify denial
NFR-A1	Architecture	Layered Architecture	Architectural inspection of component separation
NFR-A2	Architecture	Deterministic Evaluation Logic	Code inspection and repeated execution consistency test
NFR-E1	Explainability	Grounded Textual Feedback	Audit generated explanations to verify reference to computed metrics

3.2.2 Standards, laws and regulations

The AugMentor system adheres to internationally recognized standards, data protection regulations, and ethical guidelines to ensure reliability, security, transparency, and responsible AI deployment.

ISO/IEC 23053:2022 — Framework for AI Systems Using Machine Learning

This standard provides guidance for managing AI systems throughout their lifecycle, covering data governance, model training, validation, deployment, and traceability. In this project, it supports responsible AI lifecycle management by ensuring that all perception models, deviation computation logic, and Vision–Language components are version-controlled and documented. Model configurations, thresholds, and scoring formulas are traceable, and evaluation outputs are deterministic and reproducible [6].

ISO/IEC 27001 — Information Security Management

This standard defines best practices for protecting information assets through confidentiality, integrity, and availability of data. Within AugMentor 2.0, compliance is addressed through secure HTTPS/TLS communication, authenticated user access, encrypted storage of uploaded practice

videos and evaluation results, and restricted administrative privileges. These measures protect user data and reduce the risk of unauthorized access or data breaches [7].

GDPR — General Data Protection Regulation (Article 5)

GDPR establishes principles for lawful processing of personal data, including data minimization, purpose limitation, and user rights protection. Since uploaded practice videos may constitute personal data, the system collects only minimal user information, processes videos strictly for evaluation purposes, and allows users to request deletion of their account and associated data. The system performs no identity inference, facial recognition, or demographic profiling [9].

ISO/IEC 25010 — Software Product Quality Model

This standard defines quality characteristics for software systems including functional suitability, reliability, usability, and maintainability. The AugMentor architecture reflects these principles through a modular layered design separating perception, evaluation, feedback, and storage components. Deterministic metric computation ensures functional suitability, while structured feedback mechanisms enhance usability and interpretability [8].

Ethical and Responsible AI Principles

Beyond formal standards, the system adheres to recognized Responsible AI principles emphasizing transparency, fairness, accountability, and human oversight. The platform functions as an assistive educational tool and does not issue formal certification decisions. All performance evaluations are derived from explicit geometric deviation metrics, and Vision–Language explanations are strictly grounded in computed values. The system avoids demographic inference, behavioral profiling, or automated high-stakes decision making.

3.3 Conceptual Model

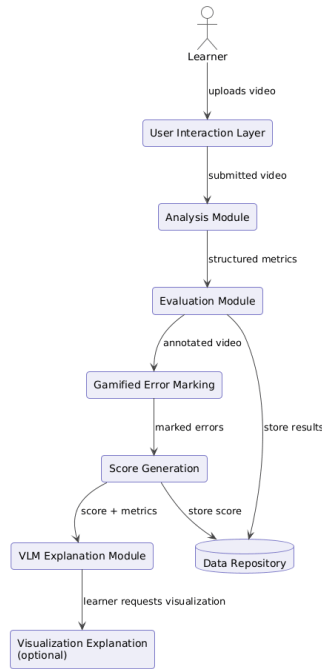


Figure 2 - Conceptual Model

Figure 2 presents the high-level conceptual model of AugMentor 2.0, represented as a block diagram using UML notation [22]. The diagram shows what the system does at each stage without committing to implementation details, which are addressed in Section 4.

3.3.1 High-Level Architectural Structure

The AugMentor system is organized into six primary conceptual modules:

1. User Interaction Layer
2. Analysis Module
3. Evaluation Module
4. Gamified Error Marking
5. VLM Explanation Module
6. Data Repository

This organization reflects the progressive processing pipeline from user input through to scored feedback and storage.

3.3.2 Description of Conceptual Modules

1. User Interaction Layer

The User Interaction Layer is the entry point of the system. It enables the learner to access courses and chapters, watch expert reference videos, upload practice videos, interact with evaluation results, and optionally request AI-generated explanations. This layer manages all learner–system interactions in accordance with the usability requirements defined in Section 3.2.

2. Analysis Module

The Analysis Module processes the uploaded video to extract structured motion and interaction data. It transforms raw video input into structured representations — including tool detections, trajectory data, angle estimates, vibration signals, and hand-to-tool distances — that serve as input to the evaluation stage.

3. Evaluation Module

The Evaluation Module compares the learner's extracted data against the expert reference and identifies deviations. It produces an annotated evaluation video marking error locations and computes the underlying deviation metrics that feed into scoring and explanation generation.

4. Gamified Error Marking

Rather than presenting results passively, the system invites the learner to watch the annotated evaluation video and actively mark the errors they perceive. This interactive step forms the basis of the scoring mechanism and reinforces perceptual awareness of correct execution.

5. VLM Explanation Module

Once a score is generated, the VLM Explanation Module generates grounded natural-language feedback strictly based on the computed deviation metrics. The learner may additionally request visual explanation overlays to complement the textual feedback. The module operates as a controlled interpretative layer and does not function as a conversational interface.

6. Data Repository

The Data Repository stores all persistent system information including course and chapter content, expert reference data, user submissions, evaluation results, and scores. It supports progress tracking, historical comparison, and compliance with data management requirements.

3.3.3 Conceptual Flow

The high-level system flow proceeds as follows:

1. The learner uploads a practice video through the User Interaction Layer.
2. The Analysis Module extracts structured motion and tool data from the video.
3. The Evaluation Module compares learner data against the expert reference and produces an annotated video.
4. The learner marks perceived errors on the evaluation video through the Gamified Error Marking interface.
5. A performance score is generated based on the accuracy of the learner's error identification.
6. The VLM Explanation Module generates grounded feedback based on the computed score and deviation metrics. The learner may additionally request visual explanation overlays.
7. All relevant results are stored in the Data Repository.

4 Solution Design

The previous section defined what AugMentor 2.0 must do. This section describes how it does it -- the architectural choices, component selection rationale, mathematical methods, and implementation-level design decisions that translate the requirements into a working system.

The solution adopts a hybrid modular architecture combining deep learning models for perception, deterministic mathematical modeling for evaluation, and a controlled Vision–Language Model layer for explanation. This approach was selected over alternatives through a structured decision process described in Section 4.1. The architecture enforces a clear separation between user interaction, motion analysis, evaluation, scoring, and explanation, satisfying NFR-A1 (Layered Architecture) and NFR-A2 (Deterministic Evaluation Logic).

The overall methodology follows three principles. First, perception is handled by machine learning models capable of operating under the visual constraints of egocentric craft video. Second, evaluation is entirely deterministic — all deviation metrics and scores are derived from explicit mathematical formulas without stochastic components. Third, explanation is generated by a Vision–Language Model strictly constrained by computed metrics, preventing hallucination and ensuring grounded pedagogical feedback in compliance with NFR-E1.

4.1 Design Alternatives and Comparison

4.1.1 High Level Alternatives

Three architectural directions were considered for the overall system design. Each was evaluated against six criteria: interpretability, data requirement, scalability, robustness, and implementation feasibility. Scores are on a scale of 1 to 5.

Alternative 1 — End-to-End Deep Learning

A single neural network predicts a performance score directly from raw video input, trained on labeled expert/novice datasets.

Advantages: high representational capacity, minimal handcrafted metric design, fully automated pipeline.

Limitations: requires large, annotated datasets unavailable for artisan craft domains; black-box decision process with no interpretable feedback; high risk of overfitting; incompatible with NFR-A2 (Deterministic Evaluation Logic), NFR-E1 (Grounded Textual Feedback), and FR-7 (Feedback Generation).

Alternative 2 — Pure Rule-Based Geometric System

All evaluation is performed through manually defined thresholds and deterministic geometric rules, with no learning-based components.

Advantages: fully interpretable, low computational cost, no training required.

Limitations: brittle under real-world variability; sensitive to noise, occlusion, and lighting conditions; poor generalization across different craft tasks and environments; insufficient robustness for NFR-R1 (Fault Tolerance) and NFR-P1 (Processing Latency).

Alternative 3 — Hybrid Modular Architecture (Selected)

Deep learning models handle perception and feature extraction. Deterministic mathematical models handle evaluation and scoring. A controlled Vision–Language Model handles explanation generation.

Advantages: balanced interpretability and adaptability; lower data requirements than end-to-end learning; modular extensibility across craft domains; grounded and reproducible explanation generation. Fully satisfies FR-6, FR-7, FR-9, NFR-A1, NFR-A2, NFR-E1, and NFR-P1.

To ensure objective selection, a weighted multi-criteria decision model was applied. Let A_j denote alternative j and w_i the normalized weight of criterion i such that $\sum_i w_i = 1$. The total score is:

$$Score(A_j) = \sum_i w_i \cdot r_{ij}$$

where $r_{ij} \in [1,5]$ is the rating of alternative j under criterion i .

Table 4 - Quantitative Decision Matrix

Criterion	Weight	End-to-End DL	Rule-Based	Hybrid (Selected)
<i>Interpretability</i>	0.25	1	5	4
<i>Data Requirement</i>	0.20	1	5	4
<i>Scalability</i>	0.15	4	2	4
<i>Robustness</i>	0.15	4	2	4
<i>Implementation Feasibility</i>	0.25	2	3	5
Weighted Score		2.05	3.40	4.20

The hybrid approach achieves the highest weighted score and is adopted as the primary architectural direction.

4.1.2 Low Level Alternatives

Following the selection of the hybrid architectural framework, each pipeline component was evaluated independently against the specific constraints of egocentric craft video. The following subsections describe the candidates tested and the rationale for each final selection.

A. Tool Detection

Off-the-shelf object detection models YOLOv8n and YOLOv8m were tested on egocentric scissors videos. Both failed to produce reliable detections, consistently misidentifying the tool as unrelated objects from their COCO training distribution — including umbrellas, surfboards, and bananas. This result confirmed that general-purpose detectors cannot be applied directly to domain-specific craft tools.

A domain-specific fine-tuning approach was adopted. A custom YOLOv8s model was trained on a labeled dataset of 584 egocentric scissors images covering diverse orientations, backgrounds, and lighting conditions. The fine-tuned model was selected as the tool detection component and serves

as the entry point for all downstream pipeline stages, providing bounding boxes used by SAM2 for tracking initialization, by the HSV angle estimator for crop extraction, and by the optical flow module for region of interest definition.

B. Blade Tip Tracking

Three approaches were evaluated for tracking the blade tip across frames.

MediaPipe hand landmark detection performed adequately under clear visibility conditions but degraded significantly in egocentric craft video, producing unstable or missing detections due to the non-standard viewpoint and partial occlusion of the hand by the tool.

Fast-HaMeR [20] produced detailed 3D hand reconstructions and demonstrated stronger robustness under difficult conditions. However, its computational cost — requiring several minutes of processing per video on available hardware — made it incompatible with NFR-P1 (Processing Latency) and impractical for a web-based evaluation platform.

SAM2 Hiera-Tiny [17] was initialized using the YOLO bounding box as a spatial prompt on the first valid detection frame. It propagated the blade tip segmentation mask across all subsequent frames, extracting the mask centroid as the blade tip position. SAM2 achieved complete tracking coverage with strong temporal stability across all evaluated videos, at a fraction of the computational cost of Fast-HaMeR. SAM2 was selected as the tracking component.

C. Blade Angle Estimation

Estimating blade orientation without relying on hand landmarks required a dedicated color-based approach operating directly on YOLO bounding box crops.

For each frame with a valid YOLO detection, the crop is converted to HSV color space. A metal mask is computed selecting pixels with saturation below 80 and value above 100, corresponding to the specular appearance of the scissor blade. A skin mask covering two hue ranges typical of skin tones is computed and subtracted, isolating the blade while removing the hand holding it. Morphological opening followed by closing reduces noise and fills gaps in the mask.

Connected contours are extracted and filtered by area (minimum 40 px²) and aspect ratio (minimum 2.0 long-to-short side), retaining only elongated blade-shaped regions. The qualifying contour with the longest axis is selected, and its orientation is estimated using OpenCV's fitLine with L2 distance, returning a unit direction vector (v_x , v_y). The blade angle is computed as:

$$\theta = \arctan 2(v_y, v_x) \bmod 180^\circ$$

mapping orientation to the undirected range $[0^\circ, 180^\circ)$, since a blade pointing left versus right represents the same physical orientation.

Temporal stability is enforced through two mechanisms applied in sequence. First, a jump filter discards any frame-to-frame angular change exceeding 35° , reusing the previous frame's angle instead. Second, accepted angles are blended through exponential smoothing with a 60% factor toward the new measurement: $\theta_{smooth}(t) = \theta_{smooth}(t-1) + 0.6 \cdot \delta$. If YOLO confidence falls below 0.35 or fitLine fails, the previous frame's angle is reused unchanged. On the first frame, the bounding box diagonal angle serves as a fallback.

Both expert and learner angle sequences — restricted to frames where a valid line was detected — are then aligned using Dynamic Time Warping (DTW) [13]. DTW finds the optimal frame-level correspondence between sequences that may differ in execution speed. Each matched pair stores the learner frame index, expert frame index, learner angle, expert angle, and absolute angle difference. An angle error is declared when the difference exceeds 15° sustained for at least 10 consecutive aligned frames.

D. Trajectory Evaluation

Trajectory evaluation is performed using a corridor-based approach. The expert blade tip trajectory is extracted offline by the administrator using the same SAM2 pipeline, producing a sequence of blade tip positions that define the reference cutting path. This trajectory is used to construct a corridor — a region of fixed margin width centered on the expert path.

At evaluation time, the corridor is rigidly translated to align its starting point with the learner's first blade tip position, preserving the corridor's orientation and shape while accounting for positional differences in setup. For each frame, the learner's blade tip position is projected onto the corridor and the perpendicular distance to the nearest corridor segment is computed. A frame is classified as outside the corridor if this distance exceeds the margin threshold. Trajectory errors are defined as contiguous windows of frames outside the corridor exceeding a minimum duration.

This approach produces spatially meaningful deviation values directly interpretable in visualization and scoring, without requiring sequence alignment.

E. Vibration Detection

Tool vibration is detected through a three-stage pipeline combining YOLO-based region extraction, RAFT optical flow estimation [21], and windowed Fast Fourier Transform analysis.

Hand vibration produces small, rapid pixel displacements typically only a few pixels per frame — a signal invisible to trajectory or angle metrics. Classical optical flow methods such as Farneback rely on coarse-to-fine pyramidal estimation and smooth out these fine displacements. RAFT, which iteratively refines a 4D correlation volume of all-pairs feature similarities through a recurrent architecture, preserves small motions significantly better. It was selected as the primary flow engine, with Farneback retained as a CPU fallback.

In Stage 1, the custom YOLOv8s model performs a pre-pass on all frames, detecting the scissors and returning bounding boxes expanded asymmetrically to include the surrounding hand region. RAFT operates exclusively on this region of interest, excluding irrelevant background motion. If detection fails on a frame, the most recent valid region is reused for up to five frames.

In Stage 2, consecutive frame pairs are preprocessed — cropped to the region of interest, converted to grayscale, triplicated to three channels to match RAFT's input format, padded to multiples of 8, and capped at 320 pixels on either dimension for practical inference latency. The model runs with automatic mixed precision in batches of 8 frame pairs. The dense flow field is reduced to per-frame scalar features including mean magnitude, maximum magnitude, motion area ratio, and direction variance.

In Stage 3, the mean magnitude time series is first differenced to suppress slow trends unrelated to vibration. A sliding one-second window of 30 samples advances in 0.5-second steps. For each window, the real FFT is computed and only frequency bins between 2 Hz and 20 Hz are considered, corresponding to the physiological range of involuntary hand tremor. The bin with maximum amplitude is recorded as the dominant frequency, and a confidence score is computed as the ratio of that bin's energy to total signal energy.

A rule-based classifier consolidates per-window results. Two conditions must both be satisfied: at least three consecutive windows must show confidence ≥ 0.15 with dominant frequency within ± 2 Hz of each other, and the locked frequency must be at least 4 Hz to exclude the deliberate rhythm of cutting motion. When both pass, severity is graded as mild (3 windows, confidence below 0.20), moderate (3 windows, confidence 0.20–0.30), or severe (4 or more windows, or confidence above 0.30).

The pipeline was validated on four test videos — one with confirmed visible hand vibration and three with steady cutting — producing correct classification on all four with no false positives or false negatives. On the vibration video, the system correctly identified the peak at 11.0 seconds at 14.48 Hz with moderate severity, matching the visually observable tremor window between 10.5 and 11.5 seconds.

The vibration verdict, severity grade, peak timestamp, and affected window list are passed to the Error Merge Service alongside trajectory and angle errors, producing a unified error model that treats vibration as a first-class error type subject to the same scoring and feedback mechanisms.

F. Hand-to-Tool Distance

The spatial distance between the non-tool hand and the tool is estimated by combining two existing detections. MediaPipe hand landmark detection provides the centroid of the non-tool hand. The YOLO bounding box center provides the tool position. The Euclidean distance between these two points is computed per frame, producing a proximity metric that captures how closely the supporting hand tracks the cutting tool — a relevant indicator of technique quality in craft tasks requiring coordinated two-hand control.

G. Vision–Language Model

VLMs were initially tested without structured input constraints. Results were inconsistent — identical video inputs produced different detailed explanations across runs, and outputs frequently included unsupported or unverifiable claims. This confirmed that unconstrained VLM usage is incompatible with NFR-E1 (Grounded Textual Feedback) and NFR-A2 (Deterministic Evaluation Logic).

The final architecture constrains the model through a structured multipart prompt described in detail in Section 4.2.1 (Figure 7). Groq's Llama-4 Scout model is used during development for its fast inference and multimodal capability. A paid production model will be used for the final deployment presentation.

The following table summarizes all component selection decisions:

Table 5 - Low-Level Component Selection

Component	Candidates Evaluated	Selected	Rationale
Tool Detection	YOLOv8n, YOLOv8m, YOLOv8s (fine-tuned)	YOLOv8s (fine-tuned)	Off-the-shelf models failed entirely; fine-tuned model achieved reliable detection
Blade Tip Tracking	MediaPipe, Fast-HaMeR, SAM2	SAM2 Hiera-Tiny	Complete tracking coverage, temporal stability, practical latency
Blade Angle Estimation	MediaPipe landmarks, HaMeR 3D mesh, HSV+fitLine	HSV + fitLine + DTW	Deterministic, lightweight, independent of hand visibility
Trajectory Evaluation	DTW on XY coordinates, Corridor geometry	SAM2 + corridor geometry	Expert path extracted by SAM2, translated into interpretable spatial corridor
Vibration Detection	Frame differencing, Farneback, RAFT	RAFT + windowed FFT	Superior accuracy on small displacements, frequency-domain vibration localization
Hand-to-Tool Distance	MediaPipe + YOLO	MediaPipe + YOLO	Direct combination of available detections, no additional model required
VLM Explanation	Unconstrained VLM, Metric-constrained VLM	Metric-constrained VLM	Eliminates hallucination, satisfies NFR-E1 and NFR-A2

4.2 Solution Modeling

4.2.1 Diagrams and Flowcharts

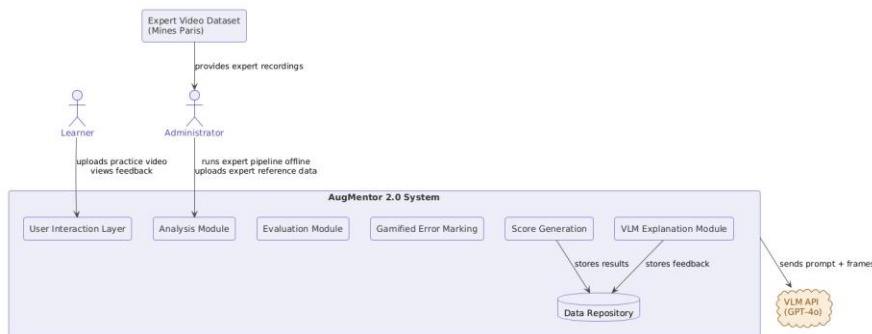


Figure 3 - System Context Diagram

The system context diagram defines the system boundary and identifies the main external actors. The Learner interacts with the platform through the web interface, uploading practice videos and receiving evaluation results, scores, and feedback. The Administrator manages course content and is responsible for uploading expert reference videos and running the expert analysis pipeline offline — pre-computing expert corridor and angle reference data so that these are available at evaluation

time without reprocessing. The system communicates with an external VLM API for explanation generation. The expert video dataset originates from the Mines Paris repository.

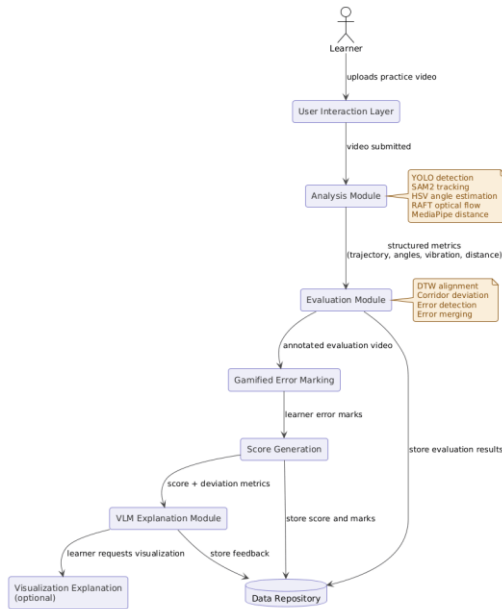


Figure 4 - High-Level Architecture Diagram

This diagram presents the overall system architecture and the flow between its primary modules. The processing flow proceeds as follows: Learner uploads a video through the User Interaction Layer → Analysis Module extracts structured data → Evaluation Module compares against expert reference and produces an annotated video → Gamified Error Marking presents the video to the learner for interactive error identification → Score Generation computes a performance score → VLM Explanation Module generates grounded feedback → optionally the learner requests a Visualization Explanation. All results are stored in the Data Repository. The diagram illustrates the central role of the Evaluation Module, the separation between perception and scoring, and the use of computed metrics to constrain VLM output.

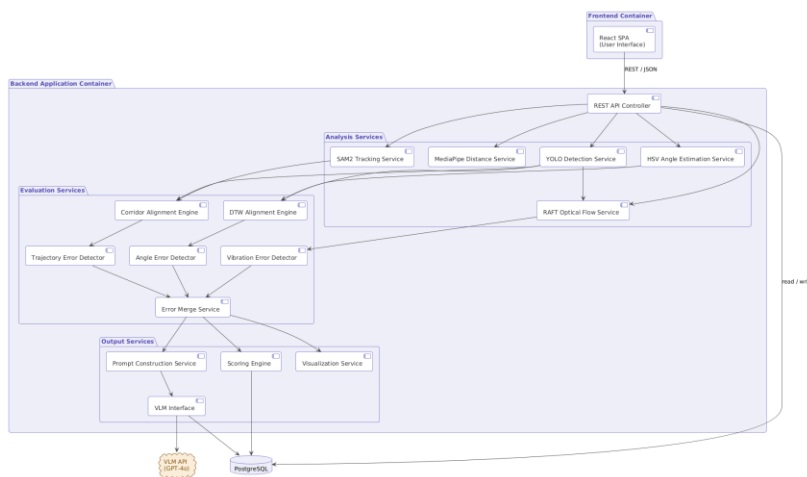


Figure 5 - Component Diagram

The component diagram decomposes the system into its implementation-level components organized across two containers. The Frontend Container is a React Single Page Application. The Backend Application Container includes the following services: REST API Controller, YOLO Detection Service, SAM2 Tracking Service, HSV Angle Estimation Service, DTW Alignment Engine, Corridor Alignment Engine, Trajectory Error Detector, Angle Error Detector, RAFT Optical Flow Service, MediaPipe Distance Service, Error Merge Service, Scoring Engine, Visualization Service, Prompt Construction Service, and VLM Interface. The system relies exclusively on pretrained or fine-tuned models operating in inference mode. A PostgreSQL database handles persistent storage of user data, evaluation metadata, and results.

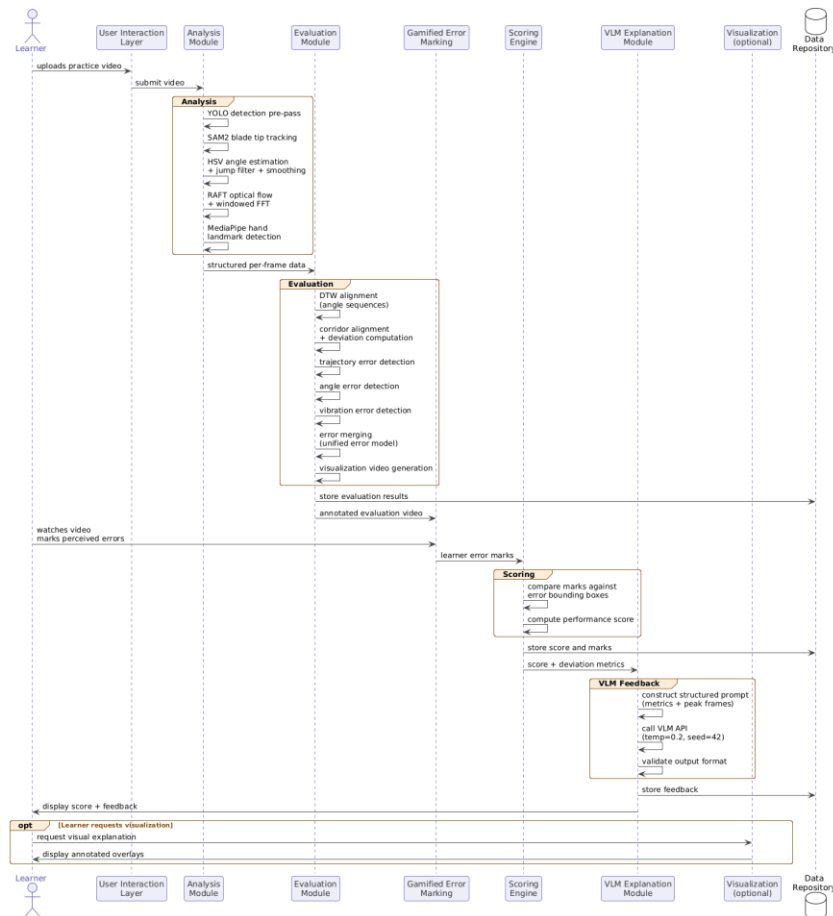


Figure 6 - Sequence Diagram

This sequence diagram models the runtime evaluation flow from video upload to feedback delivery. The learner uploads a video through the User Interaction Layer. The backend executes the Analysis Module — YOLO detection pre-pass, SAM2 blade tip tracking, HSV angle estimation with jump filtering and smoothing, RAFT optical flow computation, and MediaPipe hand landmark detection — producing structured per-frame data. The Evaluation Module then runs DTW alignment of angle sequences, corridor alignment and deviation computation, trajectory error detection, angle error detection, and error merging into a unified error model. An annotated visualization video is generated with corridor overlays and freeze-frame error annotations. The learner watches the annotated video and marks perceived errors through the Gamified Error Marking interface. The Scoring Engine compares submitted marks against detected error bounding boxes and computes a performance score. The VLM Explanation Module generates grounded pedagogical feedback. Optionally, the learner requests visual explanation overlays. All results are stored in the Data Repository.

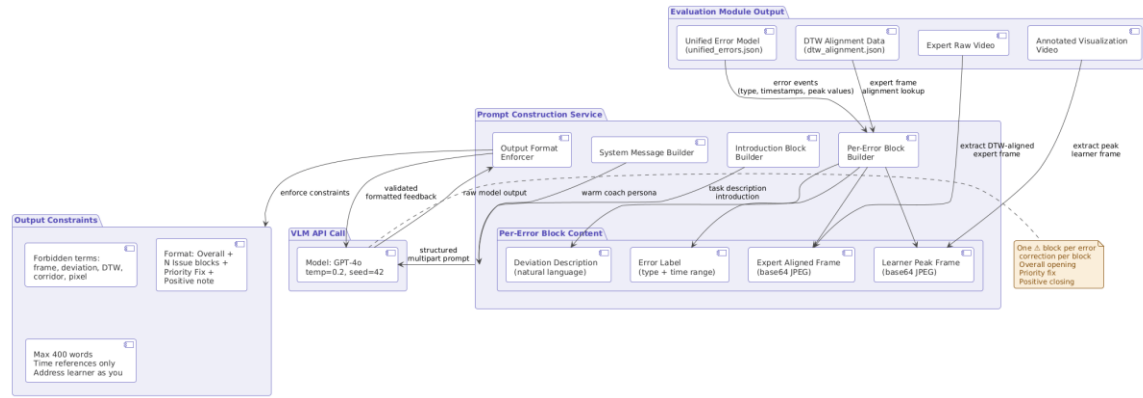


Figure 7 - VLM Prompt Construction Diagram

This diagram illustrates the methodological fusion at the heart of the explanation module. The Prompt Construction Service assembles a structured multipart message from two sources: computed deviation metrics from the evaluation pipeline, and visual frames extracted from the annotated visualization video and the raw expert video.

The system message establishes the model persona as a warm, encouraging craft coach that speaks directly to the learner, uses natural language, and never sounds like a technical report.

The user message is assembled in ordered blocks. An introduction describes the task. Then for each detected error, sorted by timestamp, the following blocks are appended: a text label identifying the error type and time range; a base64-encoded JPEG of the learner's annotated visualization frame at the peak deviation moment, with corridor lines and overlays already burned in; a base64-encoded JPEG of the expert's raw video frame at the DTW-aligned timestamp; and a natural-language description of the specific deviation. For angle errors, the description states the peak angular deviation in degrees and instructs the model to identify the rotational direction of correction from the visual comparison. For trajectory errors, the description states the direction of drift and the required correction direction.

The final block enforces a rigid output format: one Overall section, one Issue block per error each followed by a corrective instruction, a Priority Fix, and a closing positive observation. Technical terms including "frame", "deviation", "DTW", "corridor", and "pixel" are explicitly forbidden. All error references must use time ranges rather than error numbers. The model is called with temperature 0.2 and seed 42 to minimize output variance and ensure reproducible feedback grounded strictly in computed results, satisfying NFR-A2 and NFR-E1.

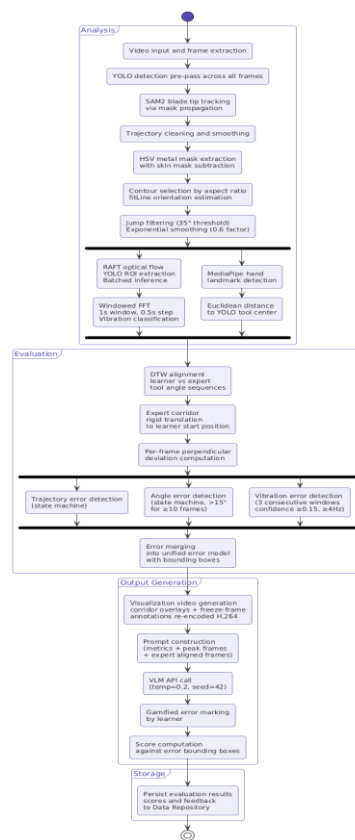


Figure 8 - Activity Diagram

This diagram represents the complete internal processing workflow from video input to stored results:

1. Video input and frame extraction
2. YOLO tool detection pre-pass across all frames
3. SAM2 blade tip tracking via mask propagation initialized from first YOLO detection
4. Trajectory cleaning and smoothing
5. HSV metal mask extraction with skin mask subtraction, contour selection by aspect ratio, and fitLine orientation estimation per YOLO crop
6. Jump filtering (35° threshold) and exponential smoothing (0.6 blend factor) of angle sequence
7. DTW alignment of valid learner and expert tool angle sequences — producing per-frame angle difference values
8. Expert corridor rigid translation to learner starting position
9. Per-frame perpendicular deviation computation against corridor centerline
10. Trajectory error detection via state machine on outside-corridor frames
11. Angle error detection via state machine on DTW-aligned frames exceeding 15° threshold
12. RAFT optical flow pre-pass — YOLO ROI extraction, batched RAFT inference on frame pairs, per-frame feature extraction
13. Windowed FFT on differenced magnitude signal — 1-second sliding window at 0.5-second steps, rule-based vibration classification with severity grading
14. MediaPipe hand landmark detection for non-tool hand centroid, Euclidean distance to YOLO tool center per frame
15. Error merging — trajectory errors, angle errors, and vibration errors consolidated into unified error model with bounding boxes

16. Visualization video generation with corridor overlays, correction arrows, and freeze-frame annotations re-encoded to H.264
17. Prompt construction and VLM call with temperature 0.2 and seed 42
18. Gamified error marking and score computation against error bounding boxes
19. Data persistence to repository.

4.2.2 Required resources

4.2.2.1 Software Resources

Table 6 - Software Resources

Category	Tool / Framework	Purpose
Programming Language	Python 3.10+	Backend pipeline and AI inference
Object Detection	YOLOv8s (custom fine-tuned)	Domain-specific scissors detection and ROI extraction
Segmentation / Tracking	SAM2 Hiera-Tiny	Blade tip trajectory extraction
Angle Estimation	OpenCV — HSV + fitLine	Blade orientation estimation per frame
Temporal Alignment	DTW (custom implementation)	Tool angle sequence alignment to expert reference
Optical Flow	RAFT (torchvision pretrained)	Tool vibration detection via frequency-domain analysis
Optical Flow Fallback	OpenCV Farneback	CPU fallback for environments without GPU
Hand Landmark Detection	MediaPipe	Non-tool hand localization for distance estimation
Video Processing	OpenCV, FFmpeg	Frame extraction and H.264 re-encoding
Numerical Computation	NumPy / SciPy	Metric computation, FFT analysis, signal processing
Backend Framework	FastAPI	REST API and background task management
Frontend Framework	React (SPA)	User interface and Compare Studio
Database	PostgreSQL	User data and evaluation metadata storage
VLM API	Groq — Llama-4 Scout (development) / GPT-4o OpenAI (production)	Pedagogical feedback generation
Version Control	Git	Source control and configuration management

4.2.2.2 Hardware Resources

Table 7 - Hardware Resources

Resource	Requirement	Purpose
GPU	NVIDIA CUDA-compatible	YOLO, SAM2, and RAFT inference
RAM	≥ 16 GB	Video processing and model execution
Storage	≥ 200 GB	Video artifacts, JSON outputs, evaluation results
Server	Local or cloud environment	Backend execution
Internet	Stable connection	VLM API communication

4.2.2.3 Dataset and Infrastructure Resources

Table 8 - Dataset and Infrastructure Resources

Resource	Description	Purpose
Expert Video Dataset	Mines Paris repository	Reference motion sequences for corridor and angle computation
Learner Test Videos	Recorded egocentric videos	Pipeline evaluation and benchmarking
Evaluation Artifacts	JSON, images, annotated video	Per-run results storage and comparison
Database Storage	PostgreSQL	Persistent storage of metrics, scores, and user data

5 Implementation, Test, Evaluation and Iterative Improvements

5.1 Technical Environment and Development Setup

The development of AugMentor 2.0 required a carefully configured technical environment capable of running multiple GPU-intensive inference models concurrently, processing full-resolution egocentric video, and hosting a multi-service web application — all within the constraints of a local development machine rather than cloud infrastructure.

Hardware Configuration

The primary development and evaluation machine was equipped with an NVIDIA GeForce RTX 3060 Laptop GPU (6 GB VRAM), an Intel Core i7 processor, and 16 GB of system RAM. This GPU served as the exclusive inference device for all deep learning models throughout the project, including the custom YOLOv8s detector, SAM2 Hiera-Tiny tracker, and RAFT optical flow estimator. All models were loaded from local weight files and executed using CUDA via PyTorch, with automatic mixed precision (AMP) enabled during RAFT batched inference to reduce memory pressure and improve throughput.

A secondary machine equipped with an NVIDIA GeForce GTX 1650 (4 GB VRAM) was used for cross-hardware validation. During the Phase 1 model exploration stage, this machine proved particularly informative for benchmarking SAM2's behaviour across GPU configurations, where a threefold difference in processing time was observed between the two GPUs on identical 10-second sequences. For the final integrated pipeline, the RTX 3060 remained the primary execution device throughout.

Storage requirements were non-trivial. Each evaluation run generates multiple JSON output files, an annotated H.264 video, and extracted frame artefacts, collectively ranging from 50 MB to over 200 MB per run depending on video length. A local SSD with approximately 500 GB of available capacity was provisioned for the project storage directory.

Table 9 - Hardware configurations used during development

Resource	Primary machine	Secondary machine
GPU	NVIDIA RTX 3060 Laptop (6 GB VRAM)	NVIDIA GTX 1650 (4 GB VRAM)
RAM	16 GB	16 GB
CUDA version	11.8	11.4
Primary role	All pipeline inference and evaluation	Cross-hardware validation

Python Environment and Dependency Management

The backend was developed entirely in Python 3.10, chosen for its compatibility with all required libraries and its stable asyncio support, which FastAPI leverages for concurrent background task execution. A dedicated virtual environment was maintained throughout the project using pip, with a pinned requirements.txt ensuring reproducibility across both machines. All inference models — YOLOv8s, SAM2, RAFT, and MediaPipe — ran fully locally; the only external network call in the pipeline was to the VLM API for explanation generation. Key dependencies and their roles are summarised in Table 10 below.

Table 10 - Key Python dependencies and roles

Package	Role in the pipeline
torch / torchvision (CUDA 11.8)	GPU inference runtime for all deep learning models
ultralytics 8.x	Loads and runs the custom YOLOv8s model (best.pt) locally
segment-anything-2	Loads and runs SAM2 Hiera-Tiny (sam2_hiera_tiny.pt) locally
opencv-python 4.x	Frame extraction, HSV masking, and fitLine angle estimation
mediapipe 0.10.x	Local non-tool hand landmark detection
numpy / scipy	Signal processing, windowed FFT, and metric computation
fastapi / uvicorn	REST API and asynchronous background task management
ffmpeg-python	H.264 video encoding for annotated output videos
psycopg2	PostgreSQL database interface
openai / groq	External VLM API communication (Groq in development, GPT-4o in production)

Model Weight Files

Three of the four core perception models are loaded from local .pt weight files at runtime, with no network dependency during inference. The custom YOLOv8s model (best.pt) was fine-tuned specifically for scissors detection and is loaded via the ultralytics library. SAM2 Hiera-Tiny (sam2_hiera_tiny.pt) is loaded via the segment-anything-2 library. RAFT optical flow weights are loaded from torchvision's pretrained model store, cached locally after the first download. MediaPipe hand landmark detection also runs fully on-device. Only the VLM explanation module makes an outbound HTTPS call, to either the Groq API (Llama-4 Scout) during development or the OpenAI API (GPT-4o) in production.

Application Stack

The backend was structured as a FastAPI application running under Uvicorn. Long-running inference tasks — which can span several minutes for a full evaluation run — were offloaded to FastAPI's BackgroundTasks mechanism, allowing the REST API to respond immediately to client requests while processing continued asynchronously. This prevented HTTP timeout failures during computationally intensive stages such as SAM2 mask propagation across several hundred frames or batched RAFT inference over full video sequences.

The frontend was implemented as a React Single Page Application built with Vite. It communicated with the backend exclusively through REST endpoints, with video playback handled natively by the browser and all evaluation results fetched as structured JSON. The Compare Studio — the interactive component displaying the annotated learner video alongside per-error overlay markers — was implemented as a custom React component with canvas-based overlays synchronised to the HTML5 video timeline.

PostgreSQL served as the persistent data store, managing user accounts, session metadata, expert reference records, and per-run evaluation results. Binary artefacts (videos, JSON outputs, preview images) were stored on the local file system under a structured directory tree and referenced in the database by path, keeping the schema minimal and queries efficient.

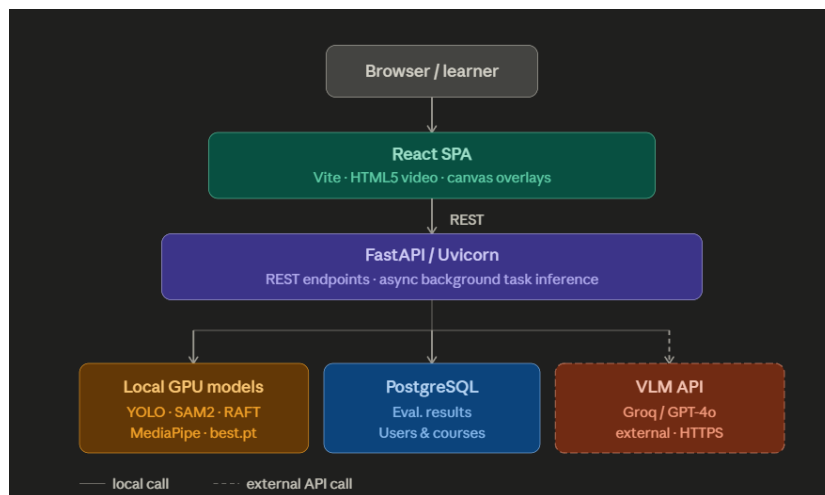


Figure 9 - AugMentor 2.0 application stack

Version Control and Development Workflow

Source code was managed using Git, with commit messages organised around pipeline components and development phases. The project followed a three-phase structure — model

exploration and benchmarking, pipeline implementation, and integration and validation — reflected in both the commit history and the directory organisation. The backend and frontend were maintained as separate top-level directories within a single repository, sharing configuration and environment files at the root level. Model weight files were stored outside the repository as local binary artefacts and loaded at runtime from a configured model directory, keeping the repository lightweight and free of large binary files.

5.2 Development Methodology

The development of AugMentor 2.0 followed a structured incremental methodology, organised around successive deliverable increments that built progressively toward a fully integrated evaluation pipeline. Rather than defining rigid sprint boundaries upfront, the process advanced increment by increment — each one producing a testable, validated artefact before the next began — ensuring that architectural decisions were grounded in empirical results at every stage.

Increment 1 — Research and Component Selection

The first increment established the theoretical and technical foundation of the system. A structured review of the literature on motion analysis, temporal alignment, optical flow, segmentation-based tracking, and Vision–Language Models identified the candidate approaches most suited to the constraints of egocentric craft video evaluation. This produced a shortlist of models and techniques to be empirically evaluated, along with the criteria — detection stability, tracking coverage, geometric interpretability, and computational feasibility — against which each would be assessed. The output of this increment was a documented component selection framework that directly governed all subsequent implementation decisions.

Increment 2 — Model Evaluation and Benchmarking

The second increment evaluated each candidate model under the specific conditions of the target domain: close-up egocentric viewpoints, partial tool occlusion, and domain-specific tooling absent from general-purpose training datasets. Models were tested on the same expert and learner video pairs and assessed against the criteria established in Increment 1. This phase revealed that no general-purpose perception model was viable without domain-specific adaptation, leading directly to the decision to fine-tune a custom YOLOv8s detector on a purpose-built annotated dataset of scissors images. The fine-tuned model achieved a mAP@0.5 of 0.995 and an F1 score of 1.00, providing the stable tool localisation foundation on which all downstream components depended.

Increment 3 — Parallel Error Branch Development

With the core detector validated, the pipeline was developed across four parallel branches, each targeting a distinct error type: trajectory deviation, blade angle error, tool vibration, and hand-to-tool distance. Each branch was implemented and tested independently against its own acceptance criteria before being submitted for integration. This parallel structure isolated failure modes at the component level, prevented cross-component interference, and allowed each branch to be refined — through parameter tuning, threshold calibration, and edge case handling — without affecting the stability of the others.

Increment 4 — Integration and End-to-End Validation

The final increment merged the validated branches into a unified backend pipeline, connected it to the FastAPI application layer and the React frontend, and validated the complete system on a straight-line cutting task. Integration testing surfaced boundary conditions that had not been visible

during isolated branch testing — corridor alignment edge cases, error merging overlap behaviour, and VLM prompt consistency — each of which was resolved before the system was accepted as complete. The validated system produced consistent, reproducible outputs across repeated runs on identical inputs, satisfying the deterministic evaluation requirement established in the system specifications.

5.3 Phase 1 — Model Exploration and Benchmarking

This phase evaluated all candidate perception approaches under the specific conditions of egocentric craft video before any pipeline integration decision was made. Each model was assessed on the same expert and learner video pairs using consistent criteria: detection stability, tracking coverage across frames, geometric interpretability of outputs, and computational feasibility on the available hardware. The following subsections document both the models selected for the final pipeline and those evaluated and set aside, with depth of coverage reflecting their role in the delivered system.

5.3.1 Custom YOLOv8s — Tool Detection

Motivation

Tool detection was the first and most critical component to resolve, since every downstream stage — SAM2 tracking initialisation, HSV angle estimation, RAFT optical flow region of interest, and MediaPipe distance computation — depended on a reliable, frame-level bounding box around the scissors. Initial experiments with the off-the-shelf YOLOv8n and YOLOv8m variants, pre-trained on the COCO dataset, produced entirely unusable results on the target domain. Rather than detecting scissors, both models consistently returned irrelevant predictions — generic object classes such as "umbrella", "surfboard", and "banana" with low confidence scores and no consistent spatial correspondence to the tool in the scene. This outcome was expected given that craft scissors held at close range in an egocentric viewpoint share almost no visual similarity with the tool representations present in COCO. A domain-specific solution was therefore necessary before any further pipeline development could proceed.

Dataset and Training

A purpose-built annotated dataset was assembled from frames extracted across the available expert and learner egocentric videos, capturing the scissors across a range of viewpoints, lighting conditions, occlusion states, and cutting positions. Images were annotated with tight bounding boxes around the scissor blade and handle region. The YOLOv8s architecture was selected over the lighter YOLOv8n and heavier YOLOv8m variants as it offered the best trade-off between detection accuracy and inference latency on the RTX 3060. The model was fine-tuned from ImageNet-pretrained weights using the ultralytics training library, with the resulting best-performing checkpoint saved as best.pt for all subsequent pipeline use.

Figure 10 shows a sample of the training data, illustrating the diversity of egocentric viewpoints, hand positions, and background conditions captured in the dataset. Figure 11 shows the corresponding validation batch with model predictions overlaid, confirming that the trained model correctly localised the scissors across the full range of visual conditions present in the data.



Figure 10 - Training batch sample



Figure 11 - Validation batch sample

The label distribution in Figure 12 confirms that the dataset captured a realistic and diverse spread of bounding box positions and aspect ratios. Box centres are concentrated in the central-right region of the frame, consistent with right-handed egocentric cutting tasks, and the aspect ratio distribution reflects the elongated shape of the scissor blade across different cutting angles.

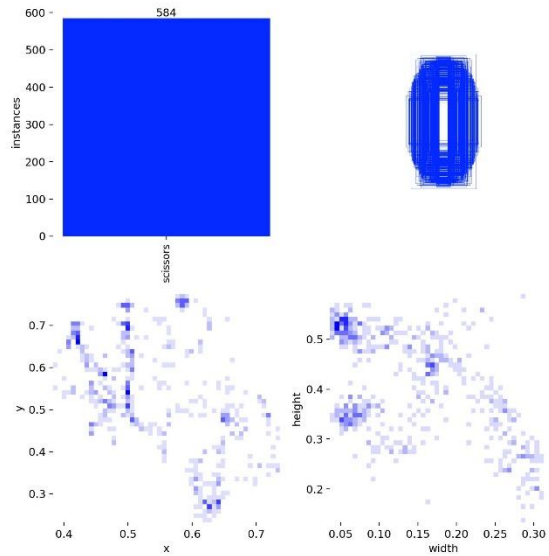


Figure 12 - Dataset label distribution

Training Results

Figure 13 presents the full set of training and validation curves across all loss components and evaluation metrics. Training converged cleanly across box loss, classification loss, and distribution focal loss on both the training and validation sets, with no evidence of overfitting. All three loss curves decreased monotonically and stabilised by the final epoch, with validation losses tracking closely behind their training counterparts throughout. Precision and recall both reached 1.00 and mAP@0.5 stabilised at 0.995 within the first quarter of training epochs, after which all metrics remained flat, indicating the model had fully saturated the task given the dataset.

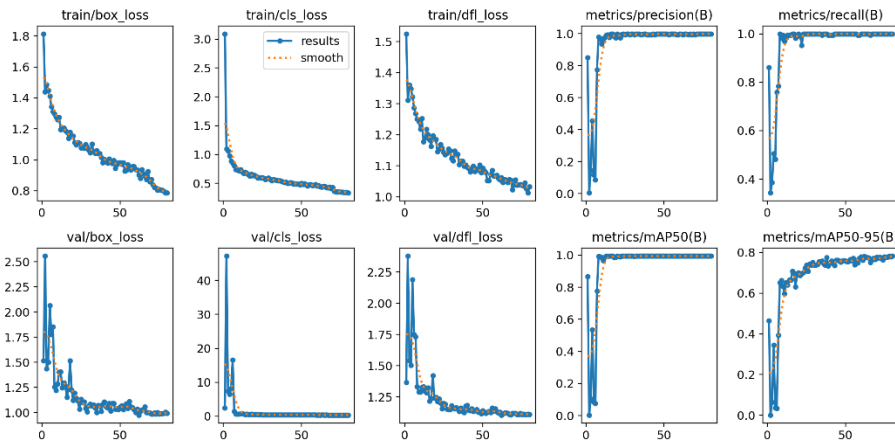


Figure 13 - Training and validation loss curves and metric progression across epochs

Evaluation Metrics

The figures below present the complete evaluation metric curves for the trained model on the held-out validation set. The F1-confidence curve in Figure 14 sustained a score of 1.00 across a wide confidence range before dropping sharply above 0.80, confirming a robust operating region well suited to the default inference threshold and indicating that the model maintained both precision and recall simultaneously across a broad range of confidence values.

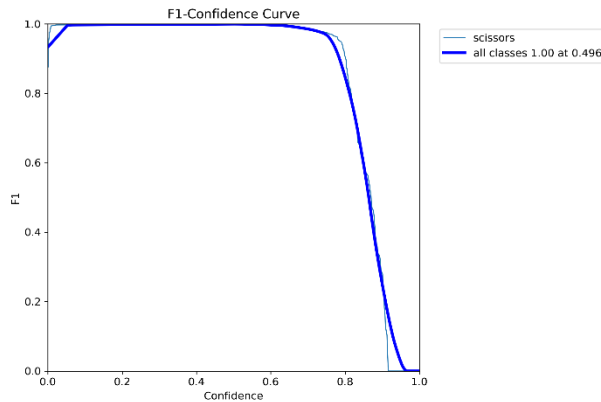


Figure 14 - F1-confidence curve

The precision-confidence curve in Figure 15 shows precision reaching and holding at 1.00 from a confidence threshold of 0.586, confirming that above this threshold the model produced no false positive detections. The recall-confidence curve in Figure 16 shows recall holding at 1.00 down to a confidence threshold of 0.000, confirming that the model missed no true positive instances across the full evaluation set.

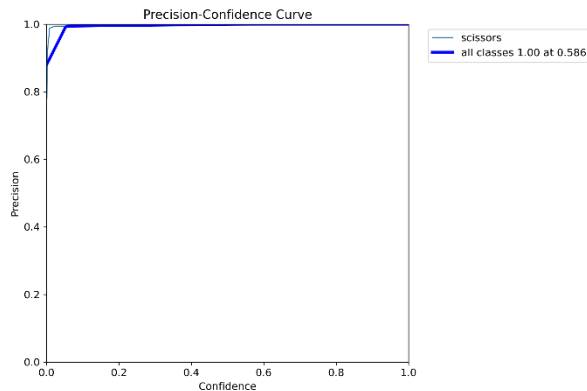


Figure 15 - Precision-confidence curve

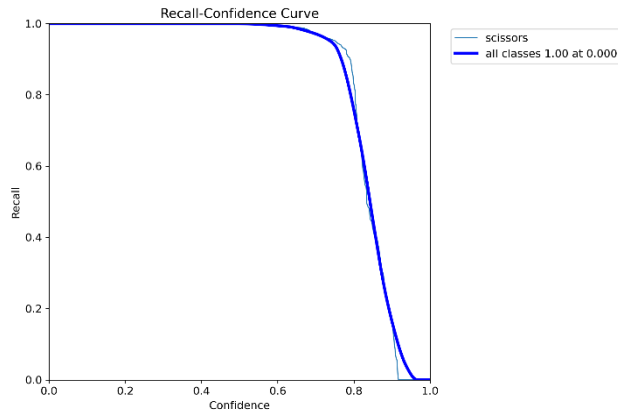


Figure 16 — Recall-confidence curve

The precision-recall curve in Figure 17 confirms a near-perfect area under the curve, with the mAP@0.5 score of 0.995 reflecting the model's ability to maintain simultaneously high precision and recall across all operating points. The normalised confusion matrix in Figure 18 shows a 1.00 classification score for the scissors class with zero background misclassifications, confirming that the model produced no false alarms on background frames throughout the evaluation set.

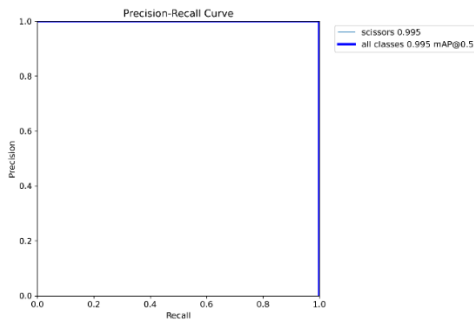


Figure 17 - Precision-recall curve

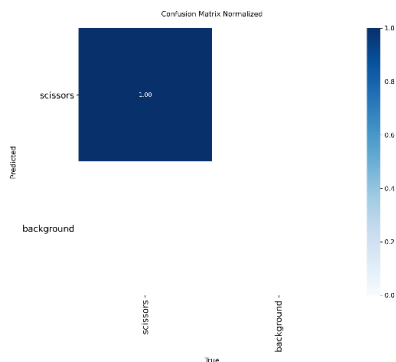


Figure 18 - Normalised confusion matrix

The final validation results are summarised in Table 11 below.

Table 11 - YOLOv8s validation results

Metric	Value
mAP@0.5	0.995
mAP@0.5:0.95	~0.77
Precision	1.00 (at confidence 0.586)
Recall	1.00 (at confidence 0.000)
F1 score	1.00 (at confidence 0.496)
True positives	166 / 166
False positives	0
False negatives	0

Role in the Pipeline

At inference time, best.pt is loaded locally via the ultralytics library and executed as a pre-pass over all frames of the learner video before any other pipeline stage begins. The returned bounding boxes, expanded asymmetrically to include the surrounding hand region, serve as the region of interest for RAFT optical flow and as the initialisation prompt for SAM2 on the first frame. On the expert side, the same pre-pass is executed once offline by the administrator when a new reference video is registered, and the results are stored for reuse across all subsequent learner evaluations against that expert.

5.3.2 SAM2 Hiera-Tiny — Blade Tip Tracking

Motivation

Once the scissors were reliably detected per frame, the next requirement was extracting the precise position of the blade tip across the full video sequence — the spatial signal from which the learner's cutting trajectory is reconstructed and compared against the expert corridor. Landmark-based approaches such as MediaPipe and Fast-HaMeR were considered but ruled out for this purpose: MediaPipe provides hand keypoints rather than tool geometry, and Fast-HaMeR's runtime of approximately 200 seconds for a 23-second video made it incompatible with an online evaluation pipeline. SAM2 Hiera-Tiny was selected as the tracking solution due to its ability to propagate a precise segmentation mask across frames from a single initialisation prompt, operating directly on the tool region without any assumption about hand visibility or landmark structure.

Implementation

SAM2 is initialised on the first frame of the video using the bounding box returned by the YOLO pre-pass as a spatial prompt. From this initialisation, the model propagates a segmentation mask forward through the video using its memory-based architecture, maintaining track of the blade region even through partial occlusions and rapid motion. The blade tip is extracted per frame as the geometrically consistent extremity of the scissors during a downward cutting motion — the lowest point of the segmentation mask within the YOLO crop. The resulting sequence of tip positions forms the raw learner trajectory, which is subsequently cleaned and smoothed before corridor comparison.

The sam2_hiera_tiny.pt weight file is loaded locally at runtime. On the expert side, SAM2 is run once offline by the administrator to produce the reference trajectory stored as the corridor centreline

for all future learner comparisons. On the learner side, SAM2 runs as part of the full evaluation pipeline triggered on each video upload.

Figure 19 shows the SAM2 segmentation mask overlaid on an egocentric frame, illustrating the precision with which the model isolates the blade region within the YOLO crop and localises the tip position used as the trajectory sample point.



Figure 19 - SAM2 segmentation mask

Figure 20 presents the tracking quality metrics for both the expert and learner videos. SAM2 achieved complete tracking coverage across all evaluated videos, with a detection rate of 100% on both sequences and no dropped frames requiring fallback handling. Temporal stability — measured as frame-to-frame consistency of the mask centroid — reached 0.802 on the expert video and 0.949 on the learner video, reflecting the smoother cutting pattern of the learner in the evaluated run.

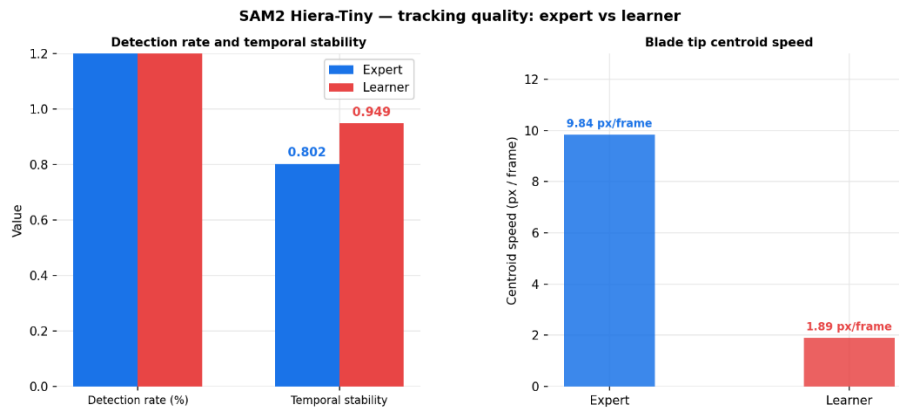


Figure 20 - SAM2 tracking quality

Table 12 - SAM2 tracking quality metrics

Metric	Expert	Learner
Detection rate	100%	100%
Temporal stability	0.802	0.949
Centroid speed (px/frame)	9.84	1.89

5.3.3 HSV + fitLine — Blade Angle Estimation

Motivation

Estimating the orientation of the scissor blade per frame required a method that operated independently of hand visibility — since the hand frequently occluded keypoint-based detectors in egocentric video — and produced a geometrically interpretable scalar output suitable for DTW alignment and threshold-based error detection. A colour-space approach combining HSV metal masking with OpenCV's fitLine function was designed for this purpose. The approach is deterministic, lightweight, and requires no additional model weight files beyond the YOLO crop already available at this stage of the pipeline.

Implementation

For each frame with a valid YOLO detection, the scissor crop is converted to HSV colour space. A metal mask is computed by selecting pixels with saturation below 80 and value above 100, targeting the specular reflectance of the blade surface. A skin mask covering two hue ranges typical of skin tones is subtracted, isolating the blade while removing the hand holding it. Morphological opening followed by closing reduces noise and fills gaps in the resulting mask. Connected contours are extracted and filtered by minimum area (40 px²) and aspect ratio (minimum 2.0 long-to-short axis), retaining only elongated blade-shaped regions. The qualifying contour with the longest axis is selected, and its orientation is estimated using fitLine with L2 distance minimisation, returning a unit direction vector (vx, vy). The blade angle is computed as:

$$\theta = \arctan2(vy, vx) \bmod 180^\circ$$

mapping orientation to the undirected range [0°, 180°), since a blade pointing left versus right represents the same physical orientation. Temporal stability is enforced through two mechanisms applied in sequence. A jump filter discards any frame-to-frame angular change exceeding 35°, reusing the previous frame's angle instead. Accepted angles are then blended through exponential smoothing with a 60% factor toward the new measurement. If the YOLO confidence falls below 0.35 or fitLine fails, the previous frame's angle is reused unchanged.

Figure 21 illustrates the angle estimation output on a representative frame, showing the expert reference angle line and the learner estimated angle line overlaid simultaneously on the scissors, with the visible divergence between the two corresponding to the detected angular deviation.

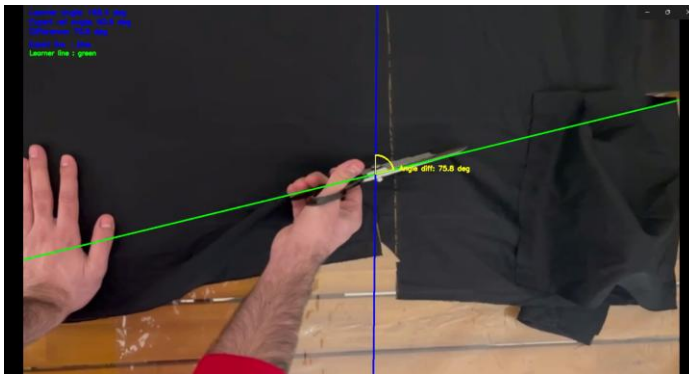


Figure 21 - Blade angle estimation

DTW Alignment

The per-frame angle sequences for the expert and learner are aligned using Dynamic Time Warping [13], which finds an optimal frame-level correspondence between sequences that may differ in execution speed. Each matched pair records the learner frame index, expert frame index, learner angle, expert angle, and absolute angular difference. An angle error is declared when the difference exceeds 15° sustained for at least 10 consecutive aligned frames.

Figure 22 presents the DTW-aligned angle comparison between the expert and learner across the full sequence, with detected error windows highlighted. The two angle error events identified in the evaluation run are clearly visible as sustained divergences from the expert reference, corresponding to the error events reported by the angle error detector.

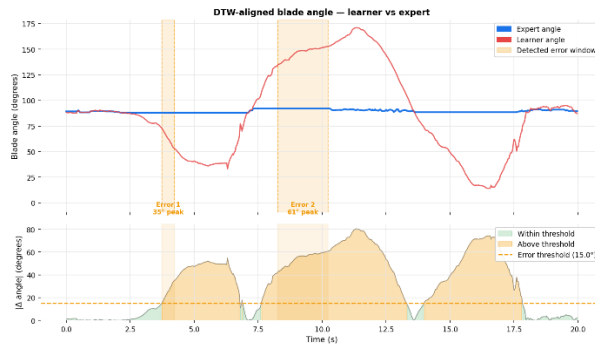


Figure 22 - DTW-aligned angle comparison

5.3.4 RAFT Optical Flow — Vibration Detection

Motivation

Tool vibration — small, rapid involuntary oscillations of the hand during cutting — is a meaningful indicator of technique quality that is invisible to both the trajectory and angle metrics. Trajectory deviation operates at the scale of centimetres of spatial drift over multiple seconds, and angle estimation works at the scale of orientation changes between frames. Neither captures the high-frequency micro-displacements of a few pixels per frame that characterise hand tremor. A dedicated signal processing approach was therefore required.

Classical optical flow methods such as Farneback were considered but ruled out for this purpose. Their coarse-to-fine pyramidal estimation architecture is designed to handle large inter-frame displacements and applies Gaussian smoothing that suppresses precisely the fine, rapid displacements that vibration produces. RAFT [21] — Recurrent All-Pairs Field Transforms — was selected instead. Its iterative refinement of a 4D correlation volume over all-pairs feature similarities preserves small motions significantly better than pyramid-based approaches, making it well suited to detecting sub-pixel-scale oscillations. An OpenCV Farneback implementation was retained as a CPU fallback for environments without GPU access, but RAFT was used for all evaluation runs on the development hardware.

Implementation

The vibration detection pipeline operates in three stages. In the first stage, the custom YOLOv8s model performs a detection pre-pass on all frames, returning bounding boxes that are expanded asymmetrically to cover the surrounding hand region. RAFT operates exclusively within this region of interest, excluding irrelevant background motion from the flow computation. If detection fails on a frame, the most recent valid region is reused for up to five frames before falling back to full-frame processing.

In the second stage, consecutive frame pairs are preprocessed — cropped to the region of interest, converted to grayscale, triplicated to three channels to match RAFT's input format, padded to multiples of eight pixels, and capped at 320 pixels on either dimension for practical inference latency. The model runs with automatic mixed precision in batches of eight frame pairs. The dense flow field is reduced per frame to a set of scalar features: mean magnitude, peak magnitude, motion area ratio, and direction variance.

In the third stage, the mean magnitude time series is first-differenced to suppress slow trends unrelated to vibration. A sliding one-second window of 30 samples advances in 0.5-second steps across the full sequence. For each window, the real FFT is computed and only frequency bins between 2 Hz and 20 Hz are considered, corresponding to the physiological range of involuntary hand tremor [15]. The bin with the maximum amplitude is recorded as the dominant frequency, and a confidence score is computed as the ratio of that bin's energy to the total signal energy in the window.

A rule-based classifier then consolidates the per-window results. Two conditions must both be satisfied for a vibration event to be declared: at least three consecutive windows must show a confidence score above 0.15 with a dominant frequency within ± 2 Hz of each other, and the locked frequency must be at least 4 Hz to exclude the deliberate rhythm of the cutting motion itself. When both conditions pass, severity is graded as mild (three windows, confidence below 0.20), moderate (three windows, confidence between 0.20 and 0.30), or severe (four or more windows, or confidence above 0.30).

Validation

The pipeline was validated on two test videos recorded under controlled conditions: one in which the operator deliberately introduced visible hand tremors during the cutting motion, and one in which the same zigzag cutting pattern was performed steadily without tremor. The two videos were processed independently, and the per-window dominant frequency and confidence scores are shown in Figure 23

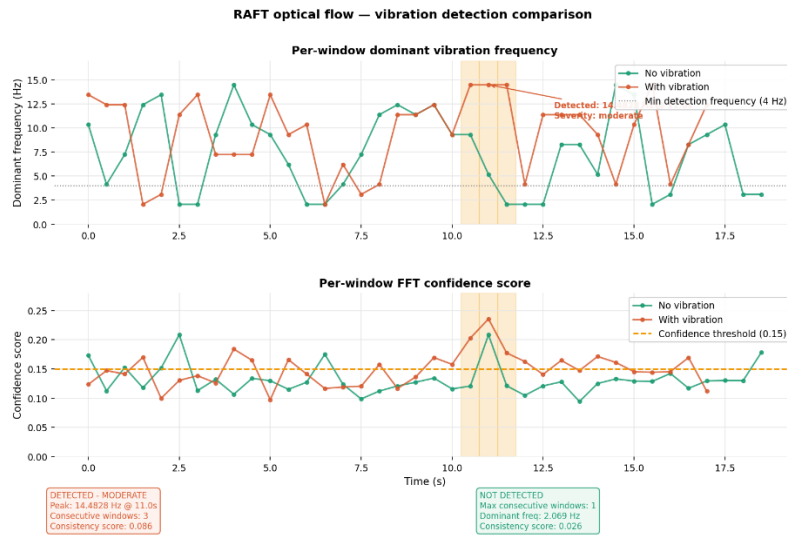


Figure 23 - vibration detection comparison

In the vibration video, the system identified three consecutive windows at 10.5, 11.0, and 11.5 seconds all locking onto 14.48 Hz with peak confidence of 0.2356 at 11.0 seconds, satisfying the detection conditions and producing a moderate severity verdict. Outside this window, the dominant frequency fluctuated widely across the 2–14 Hz range with no consecutive frequency lock, correctly indicating normal cutting motion. In the no-vibration video, no three consecutive windows achieved a consistent frequency lock above the confidence threshold. The maximum consecutive window count was 1, the consistency score was 0.026 compared to 0.086 in the vibration case, and the dominant frequency identified by the classifier was 2.07 Hz — well below the 4 Hz minimum required for a vibration declaration. The system correctly returned no detection on this video.

The key metrics comparing both videos are summarized in Table 13 below.

Table 13 - Vibration detection results

Metric	With vibration	No vibration
Vibration detected	Yes	No
Severity	Moderate	None
Peak frequency (Hz)	14.48	—
Peak timestamp (s)	11.0	—
Consecutive windows	3 (10.5–11.5 s)	1 (max)
Peak confidence	0.236	0.209 (isolated)
Consistency score	0.086	0.026
Vibration score	0.317	0.233

The validation confirmed 100% accuracy across both test cases with no false positives or false negatives, and the detected peak at 11.0 seconds matched the visually observable tremor window in the vibration video. The vibration verdict, severity grade, peak timestamp, and affected window list are passed to the Error Merge Service alongside trajectory and angle errors, where vibration is treated as a first-class error type subject to the same scoring and feedback mechanisms as the geometric errors.

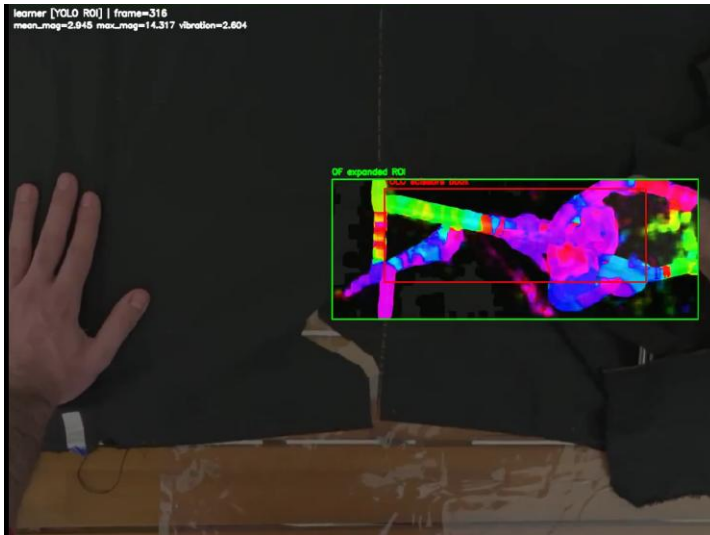


Figure 24 - RAFT optical flow

5.3.5 MediaPipe — Hand-to-Tool Distance

MediaPipe Hands was evaluated early in the project for hand landmark detection and demonstrated a detection rate of only 1.5% on the primary benchmarking videos, where the operator's hands were covered by dark gloves and long sleeves. Under these conditions the model consistently failed to localise any landmarks, making it unsuitable as a primary perception component.

However, MediaPipe was retained in the final pipeline for a more specific and appropriate role: estimating the spatial proximity of the non-tool hand to the scissors across frames. In the target cutting task, the non-tool hand guides the material being cut, and its distance from the blade is a relevant indicator of technique quality and safety. When the non-tool hand is sufficiently visible — which occurs during a meaningful proportion of frames in the learner videos used for evaluation — MediaPipe provides the hand centroid reliably and at negligible computational cost. The Euclidean distance between this centroid and the YOLO bounding box centre is computed per frame, producing a proximity metric passed to the unified error model alongside trajectory and angle data. Missing frames where detection fails are handled gracefully without affecting the core evaluation output.

5.3.6 V-JEPA 2.1 — Semantic Video Similarity

V-JEPA 2.1 [18][19] was evaluated as a high-level semantic comparison tool, operating by dividing each video into temporal segments and computing cosine similarity between corresponding segment embeddings learned through self-supervised prediction. On the evaluated expert–learner pair the model produced a final semantic similarity score of 0.544 and a mean segment similarity of 0.684. As shown in Figure 25, the model successfully identified segment 2 as the most divergent region of the learner's performance, with a similarity score of 0.458 compared to the 0.7–0.8 range observed across the remaining segments.

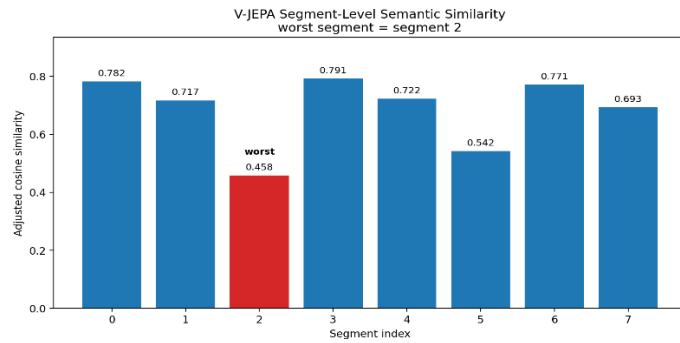


Figure 25 - V-JEPA 2.1 segment-level semantic similarity

While V-JEPA demonstrated a meaningful ability to localise temporal divergence at the segment level, its outputs lacked the geometric precision required for the pipeline's error detection and scoring mechanisms. Similarity scores between embeddings do not map to measurable quantities such as angular deviation in degrees or spatial deviation in pixels, making them unsuitable as inputs to the threshold-based error detection logic. V-JEPA was therefore not integrated into the final pipeline.

5.3.7 Fast-HaMeR — 3D Hand Reconstruction

Fast-HaMeR [20] was evaluated for its ability to reconstruct detailed 3D hand geometry from monocular RGB frames, producing dense mesh representations that include finger flexion angles, hand openness, and thumb-index spread. As shown in Figure 26, the model demonstrated strong qualitative reconstruction quality under the egocentric viewpoint conditions of the target task, with the reconstructed mesh remaining well-aligned with the visible hand region even under partial occlusion. A detection rate of 80.8% was achieved on the test videos, substantially higher than MediaPipe under the same conditions.



Figure 26 - Fast-HaMeR 3D

The critical limitation was runtime. Fast-HaMeR required approximately 200 seconds to process a 23-second video on the RTX 3060, a factor of nearly nine times real-time. This was incompatible with the requirement for online learner evaluation on each video upload, where total pipeline latency must remain within acceptable bounds. Fast-HaMeR was therefore set aside in favour of the lighter MediaPipe approach for hand localisation, with its geometric richness acknowledged as a potential avenue for future pipeline enhancement.

5.3.8 Vision–Language Model — Prompt Architecture and Explanation Generation

Unconstrained VLM Evaluation

An initial evaluation of Vision–Language Models without structured input constraints was conducted to establish a baseline for explanation quality and consistency. Two independent runs of the same expert–learner video pair were submitted to the model under identical conditions. As shown in Table 14, while the core measurements surfaced in both outputs were consistent — all four errors were identified at the correct timestamps with identical angular deviation values — the natural language used to describe corrections varied meaningfully between runs. Error 1, a 35° blade angle deviation, was described as requiring the learner to "rotate the blade to the left" in the first run and "rotate the blade counter-clockwise" in the second. The priority fix and closing positive observation also differed in emphasis across the two runs. This variability confirmed that an unconstrained VLM could not satisfy the system's requirements for deterministic evaluation logic (NFR-A2) and grounded textual feedback (NFR-E1), since the same measured error could be communicated differently — or even contradictorily — across evaluation sessions. A structured, metric-constrained prompt architecture was therefore designed.

Table 14 - Two-run VLM output comparison on identical input

Aspect	Run 1	Run 2
All 4 errors detected	Yes	Yes
Error 1 peak angle	35°	35°
Error 3 peak angle	61°	61°
Error 1 correction phrasing	"rotate 35° to the left"	"rotate 35° counter-clockwise"
Priority fix	Consistent blade angle throughout	Blade angle adjustment early on
Closing positive	Good control, smooth cuts	Good control over scissors

Prompt Architecture

The final VLM integration constrains the model entirely through a structured multipart prompt assembled by the Prompt Construction Service before each API call. The prompt is composed of ordered blocks that provide both computed numerical context and visual evidence for every detected error, ensuring that the model interprets measurement results rather than assessing the video independently.

The system message establishes the model persona as a warm, encouraging craft coach that speaks directly to the learner in natural language and never sounds like a technical report. A list of forbidden terms — including "frame", "deviation", "DTW", "corridor", and "pixel" — is explicitly enforced to prevent internal pipeline language from surfacing in the generated output.

The user message is assembled as follows. An introductory block describes the task context. For each detected error, sorted by timestamp, four blocks are appended in sequence: a text label identifying the error type and time range; a base64-encoded JPEG of the learner's annotated visualisation frame at the peak deviation moment, with corridor lines and overlays already burned in; a base64-encoded JPEG of the expert's raw video frame at the DTW-aligned timestamp; and a natural-language description of the specific deviation. For angle errors, the description states the peak angular deviation in degrees and instructs the model to identify the correction direction from the visual comparison between the two frames. For trajectory errors, the description states the direction of drift and the required correction. A rigid output format is enforced in the final block: one Overall section, one Issue block per error each followed by a corrective instruction, a Priority Fix, and a closing positive observation. The model is called with temperature 0.2 and seed 42 to minimise output variance and ensure reproducible feedback grounded strictly in the computed measurements.

The output of the constrained pipeline, shown in Figure 27 as rendered in the AugMentor 2.0 interface, demonstrates the effect of the prompt architecture. Each issue block references a specific timestamp, states the exact measured deviation, and provides a directional corrective instruction. No pipeline terminology appears in the output. The structured format is preserved exactly across runs, with variation limited to natural language phrasing of the corrective instructions — a residual variance that the temperature and seed parameters minimise but do not entirely eliminate, as demonstrated by the two-run comparison in Table 14.

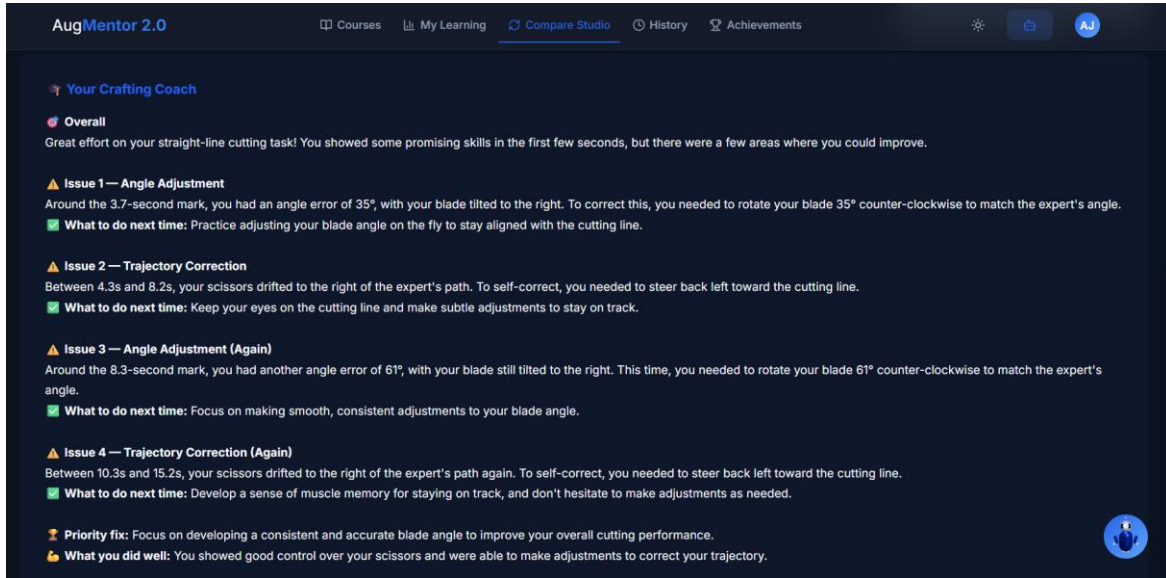


Figure 27 - VLM-generated explanation output rendered in the AugMentor 2.0

Summary of Model Evaluation

The evaluation phase assessed seven candidate approaches against the specific constraints of egocentric craft video analysis: close-up viewpoints, partial tool occlusion, domain-specific tooling, and the requirement for geometric precision sufficient to support threshold-based error detection. No single model satisfied all requirements in isolation. The final pipeline emerged from combining the strengths of complementary approaches — a custom-trained detector for reliable tool localisation, a segmentation-based tracker for trajectory extraction, a deterministic colour-space method for angle estimation, a signal-processing approach for vibration detection, and a lightweight landmark detector for hand proximity — with a metric-constrained Vision–Language Model providing the explanation layer. Table 15 summarises the evaluation outcome for all assessed models.

Table 15 - Comparative assessment of all evaluated models

Model	Accuracy / Reliability	Speed	Final role	Assessment
YOLOv8s (custom)	High	Fast	Core — ROI and detection foundation	Fine-tuned on domain dataset; mAP@0.5 = 0.995; underpins all downstream stages
SAM2 Hiera-Tiny	High — stable tracking	Medium	Core — trajectory extraction	100% tracking coverage; dominant pipeline cost; GPU mandatory
HSV + fitLine	High — deterministic	Negligible	Core — angle estimation	Lightweight; no model weights; robust to occlusion via colour masking
RAFT optical flow	High — vibration detection	Fast	Core — vibration detection	ROI-focused inference; validated on two controlled test videos
MediaPipe	Low under occlusion	Fast	Supplementary — hand distance	Reliable when hand visible; graceful fallback on missing frames
V-JEPA 2.1	Medium — semantic only	Hardware-dependent	Not used	Lacks geometric precision for threshold-based error detection
Fast-HaMeR	High — geometric detail	Slow	Not used	Strong reconstruction quality; runtime incompatible with online evaluation
VLM unconstrained	Medium — inconsistent	Medium	Not used standalone	Non-deterministic; replaced by metric-constrained prompt architecture
VLM constrained	High — grounded output	Medium	Core — explanation generation	Prompt-engineered; temperature 0.2; seed 42; forbidden terms enforced

The following section describes the implementation of each chosen component within the integrated pipeline.

5.4 Phase 2 — Pipeline Implementation

With the component selection decisions established in Phase 1, the second phase focused on implementing and connecting the individual pipeline stages into a unified processing chain. The pipeline is structured around a clear separation between expert-side processing — which runs once offline when a new reference video is registered — and learner-side processing, which runs in full on every video upload. Figure 28 presents the complete pipeline flow.

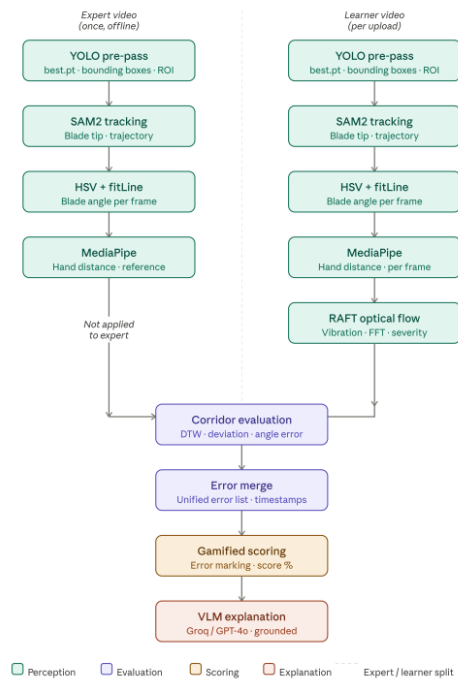


Figure 28 - AugMentor 2.0 evaluation pipeline

5.4.1 YOLO Pre-Pass and ROI Extraction

The pipeline begins with a full-video pre-pass using the custom fine-tuned best.pt model loaded locally via the ultralytics library. Every frame is processed in a single sequential pass before any other stage begins, producing a per-frame record of the scissors bounding box, confidence score, and class label. Boxes are expanded asymmetrically around the detected region — 2.4× horizontally and 2.5× vertically with an additional downward extension ratio of 1.5 — to ensure the surrounding hand and wrist area is captured within the region of interest for downstream optical flow and angle estimation. If detection fails on a frame, the most recent valid box is reused for up to five frames before the frame is flagged as a fallback. Frames flagged as fallback are excluded from angle estimation and optical flow processing but do not terminate the pipeline. On the expert side this pre-pass is run once at registration time and its outputs are stored permanently. On the learner side it runs at the start of every evaluation.

5.4.2 SAM2 Blade Tip Tracking and Trajectory Extraction

SAM2 Hiera-Tiny is initialised on the first frame of the video using the bounding box returned by the YOLO pre-pass as a spatial prompt. The model propagates a segmentation mask forward through all subsequent frames using its memory-based architecture, maintaining continuous track of the blade region. The blade tip is extracted per frame as the lowest point of the segmentation mask within the YOLO crop — the geometrically consistent extremity of the scissors during a downward cutting motion. Raw tip positions are written to a JSON file as the raw trajectory. The raw trajectory is then passed through a cleaning stage that removes outlier points where the tip displacement between consecutive frames exceeds a configurable threshold, interpolates across any short gaps, and applies Savitzky-Golay smoothing to produce the final smoothed trajectory used for all subsequent evaluation. On the expert side the smoothed trajectory forms the corridor centreline stored for future learner comparisons. On the learner side the smoothed trajectory is evaluated against the translated expert corridor at each upload.

5.4.3 HSV + fitLine Blade Angle Estimation

For each frame with a valid YOLO detection, the scissor crop is extracted and converted to HSV colour space. A metal mask is computed by thresholding pixels with saturation below 80 and value above 100, targeting the specular reflectance of the blade surface. A skin mask covering two hue ranges typical of skin tones is subtracted from the metal mask, isolating the blade while removing the hand holding it. Morphological opening followed by closing cleans noise and fills gaps in the resulting binary mask. Connected contours are extracted and filtered by minimum area and minimum aspect ratio, retaining only elongated blade-shaped regions. The qualifying contour with the longest principal axis is selected, and its orientation is estimated using OpenCV's `fitLine` function with L2 distance minimisation. The blade angle is computed as $\theta = \arctan2(vy, vx) \bmod 180^\circ$, mapping orientation to the undirected range $[0^\circ, 180^\circ)$. A jump filter discards frame-to-frame angular changes exceeding 35° , and accepted angles are blended through exponential smoothing. The resulting per-frame angle sequence is stored for DTW alignment during the evaluation stage.

5.4.4 RAFT Optical Flow — Vibration Detection

RAFT optical flow is run exclusively on the learner video, since vibration is a learner-side error rather than a reference metric. Frame pairs within the YOLO region of interest are preprocessed — cropped, converted to grayscale, padded to multiples of eight pixels, and capped at 320 pixels on either dimension — then passed through RAFT in batches of eight with automatic mixed precision enabled. The dense flow field is reduced per frame to a scalar mean magnitude. The mean magnitude time series is first-differenced to remove slow trends, then analysed through a sliding one-second window FFT advancing in 0.5-second steps. Frequency bins between 2 Hz and 20 Hz are retained and the dominant bin per window is recorded alongside its confidence score. A vibration event is declared when at least three consecutive windows lock onto the same frequency (within ± 2 Hz) with confidence above 0.15 and a dominant frequency above 4 Hz. Detected events are graded as mild, moderate, or severe based on window count and peak confidence, and passed to the error merge stage as first-class error objects with a timestamp, severity, and peak frequency.

5.4.5 MediaPipe — Hand-to-Tool Distance

MediaPipe Hands is run on both the expert and learner videos to estimate the proximity of the non-tool hand to the scissors across frames. On the expert side, the resulting per-frame distance series is stored at registration time as the reference proximity profile against which the learner's distance is compared. On the learner side, the same process runs as part of the full evaluation pipeline on each upload. For each frame where MediaPipe successfully localises at least one hand landmark set, the wrist landmark of the hand not holding the scissors is extracted. The Euclidean distance between this wrist position and the centre of the YOLO bounding box is computed in pixels and normalised by the diagonal of the YOLO box to produce a scale-invariant proximity metric. Frames

The expert smoothed trajectory is stored as the corridor centreline at registration time. When a learner video is evaluated, the corridor is translated into the learner's spatial reference frame using an anchor-based alignment: the start point of the expert centreline is matched to the start point of the learner's smoothed blade tip trajectory, and a rigid translation vector is computed. Left and right corridor edges are generated by offsetting the translated centreline by the configured margin of 80 pixels in the perpendicular direction at each point, producing a band within which the learner's trajectory is considered correct.

```
mode: show, log=0, cut=interior, margin=50px, expected_cdf=180px
px=149.7, qyr=52.7, latden=45.05
width: min=160.0, max=160.0, mean=160.0px
accepted_seg=1, rejected=0
seg1: 46877746px, dir=(-0.008,-1.000) [turns]
REJECTED seg0: 10px (length 16.4px < 40.0px; net_dpo 0.6px < 60.0px)
REJECTED seg1: 40px (length 60.1px < 80.0px; net_dpo 30.1px < 60.0px)
```

The learner trajectory is sampled at a configurable frame stride and each sampled point is checked against the nearest point on the corridor centreline. Points whose perpendicular distance to the centreline exceeds the corridor margin are flagged as outside. Contiguous runs of outside frames exceeding a minimum duration are merged into trajectory error events, each recording the start and end frame, peak deviation in pixels, mean deviation, direction of drift (left or right), and peak location.

Each of the four error branches — trajectory deviation, blade angle error, tool vibration, and hand distance — produces its own list of error events. The Error Merge Service consolidates these into a single unified error list sorted by timestamp. Overlapping or adjacent error events of the same type within a configurable merge gap are combined into a single event to avoid reporting the same deviation as multiple separate errors. Each merged event in the unified list is assigned a unique

error ID, an error type label, a timestamp range, a peak location in pixel coordinates, and a bounding box used by the gamified marking interface for spatial hit testing.

The angle error preview in Figure 30 and the trajectory error preview in Figure 31 illustrate the detected error events as rendered in the system's visualisation output, with error regions annotated directly on the video frame.

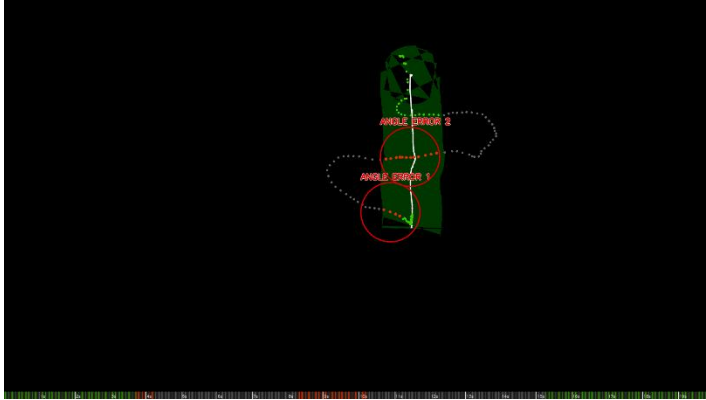


Figure 30 - Angle error preview

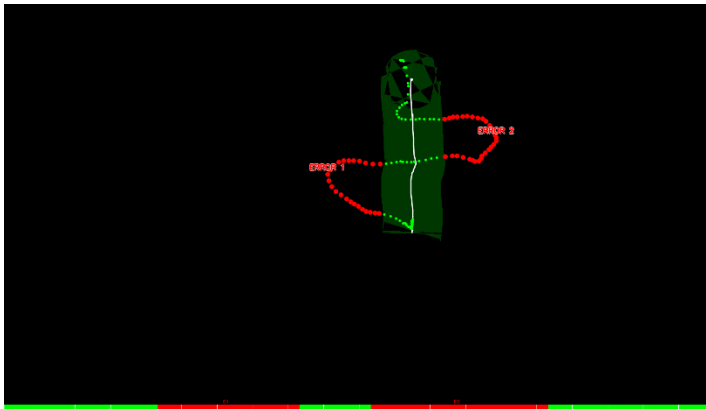


Figure 31 - Trajectory error preview

5.4.8 Gamified Error Marking and Scoring

The gamified scoring module presents the learner with the annotated evaluation video and challenges them to identify the errors they can perceive by placing marks on the video at the locations and timestamps they believe correspond to mistakes. Each mark is submitted as a type label (trajectory or angle), a pixel coordinate, and a timestamp. The scoring engine compares submitted marks against the unified error list using a spatial and temporal matching criterion: a mark is accepted as correct if its pixel coordinate falls within the bounding box of a real error event and its timestamp falls within the event's time range. Each real error can be matched by at most one mark, and each mark can match at most one real error. The final score is expressed as a percentage of real errors correctly identified, with zero false alarms counted separately. The score, correct marks, false alarms, and missed errors are stored in the evaluation record and displayed to the learner on completion.

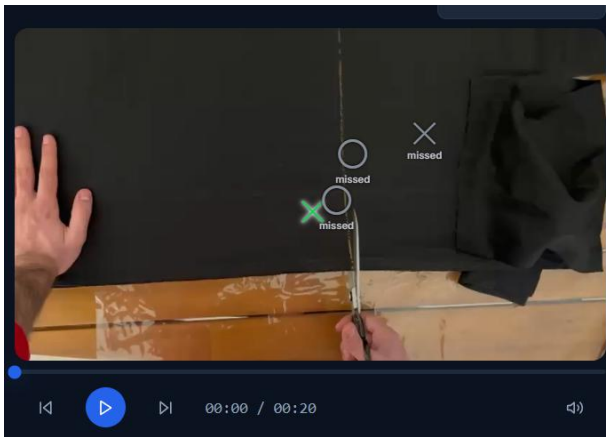


Figure 32 - Gamified error marking interface

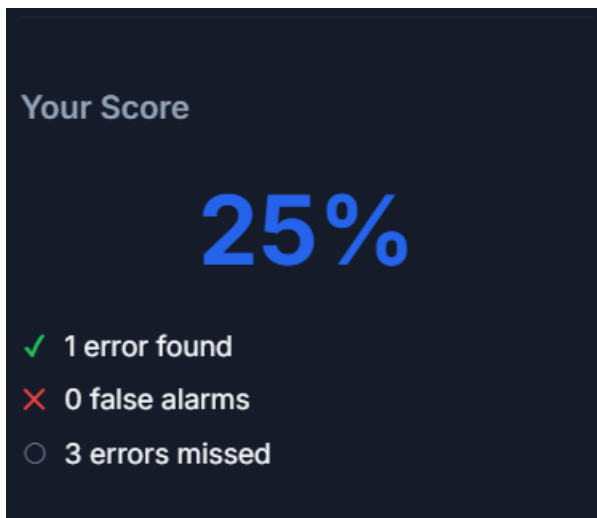


Figure 33 - Score result

5.4.9 VLM Explanation Generation

The VLM explanation stage is the final stage of the learner evaluation pipeline. The Prompt Construction Service assembles a structured multipart prompt from the unified error list and the evaluation metadata. For each error event, a peak-frame annotated JPEG of the learner video and a DTW-aligned frame from the expert video are encoded as base64 and included in the prompt alongside a natural-language description of the measured deviation. The complete prompt is submitted to the Groq API (Llama-4 Scout) during development and the OpenAI API (GPT-4o) in production, with temperature set to 0.2 and seed set to 42 to minimise output variance. The returned explanation is validated against the forbidden terms list and stored in the evaluation record. The explanation is displayed to the learner in the Compare Studio interface alongside the annotated video and score breakdown.

5.4.10 Performance Analysis

Performance evaluation was conducted across two stages. During Phase 1 benchmarking, all candidate models were profiled running on both expert and learner videos simultaneously, reflecting the naive approach where both sequences are processed together on every evaluation request. The architectural decision to pre-process the expert video offline — established during Phase 2 — eliminated the expert processing cost from the learner evaluation runtime entirely, producing a significant reduction in end-to-end latency.

Phase 1 benchmarking — dual-video processing

Table 16 summarises the runtime measurements recorded during Phase 1 for all evaluated models under their respective hardware configurations. These timings reflect full pipeline execution on both expert and learner video pairs processed together.

Table 16 - Phase 1 runtime comparison

Model / Configuration	Video length	FPS / Sampling	Runtime (s)
Fast-HaMeR (RTX 3060)	23 sec	15 FPS	200
MediaPipe (CPU)	23 sec	30 FPS	40
SAM2 (CPU)	10 sec	1 frame / 3	600+
SAM2 (RTX 3060)	10 sec	1 frame / 3	60
SAM2 (GTX 1650)	10 sec	1 frame / 3	175
YOLOv8n (CPU)	23 sec	30 FPS	61
YOLOv8m (CPU)	23 sec	30 FPS	312

Several observations emerged from these results. SAM2 running on CPU alone exceeded 600 seconds for a 10-second video sampled at one frame in three, making CPU-only deployment entirely impractical. GPU acceleration on the RTX 3060 reduced this to 60 seconds — a tenfold improvement — while the GTX 1650 required 175 seconds under identical conditions, confirming the RTX 3060 as the minimum viable GPU configuration. Fast-HaMeR required 200 seconds for a 23-second video, ruling it out for online evaluation despite its strong reconstruction quality. MediaPipe was the only model capable of processing a 23-second video in under a minute on CPU alone, which motivated its selection for the lightweight hand distance estimation role.

The figures below illustrate the resource utilisation profiles of the core perception models recorded during this phase. Figure 34 shows SAM2's CPU-only usage profile, confirming the sustained high load that makes GPU acceleration mandatory.

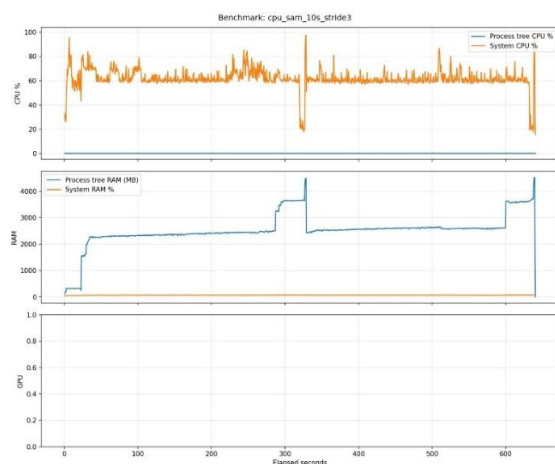


Figure 34 - SAM2 CPU-only usage

When GPU acceleration is available the load shifts from CPU to VRAM. Figure 35 shows the SAM2 memory and utilisation profile on the RTX 3060, where a characteristic step-wise VRAM loading pattern reflects batch-based frame propagation.

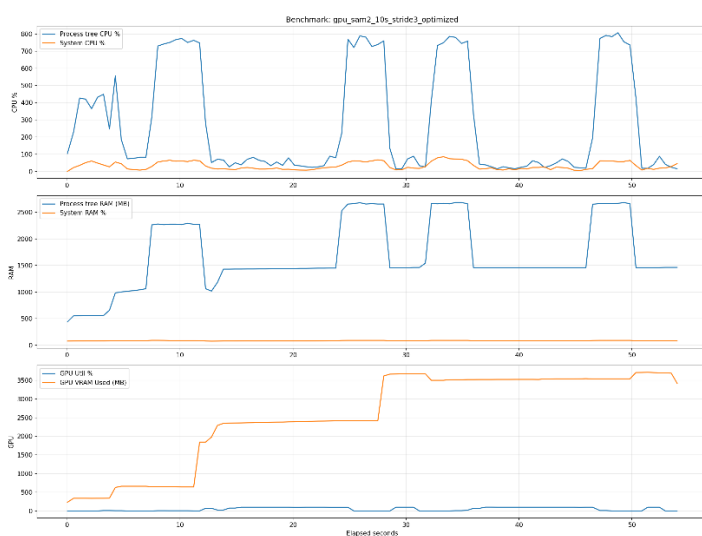


Figure 35 - SAM2 GPU memory and utilisation profile on RTX 3060

Figure 36 shows the corresponding profile on the GTX 1650, where the same pattern is observed at lower throughput, directly explaining the threefold runtime increase relative to the RTX 3060.

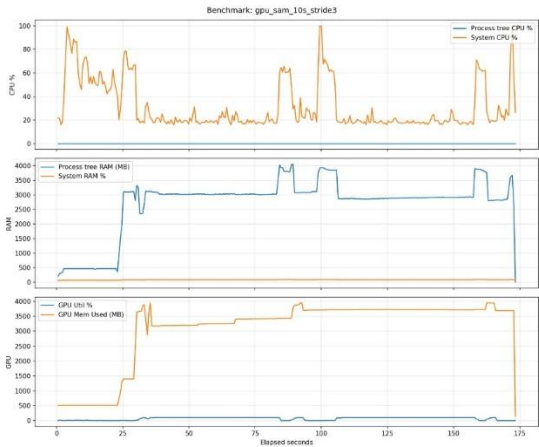


Figure 36 - SAM2 GPU memory and utilisation profile on GTX 1650

In contrast, Figure 37 confirms that MediaPipe imposes negligible CPU load throughout execution, consistent with its role as the pipeline's lightest component.

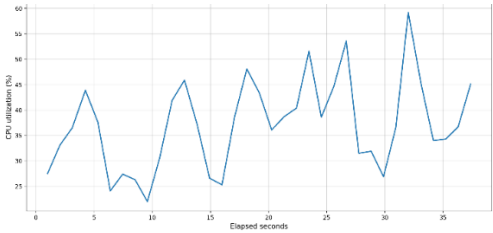


Figure 37 - MediaPipe CPU usage

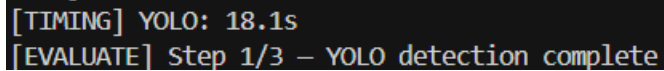
Final pipeline — learner-only online evaluation

The architectural decision to run expert processing offline once at registration time, rather than on every evaluation request, reduced the per-evaluation runtime substantially. Table 17 presents the per-stage runtime breakdown for a complete learner evaluation run on the primary development machine, measured on an 18-second learner video at 30 fps.

Table 17 - Per-stage runtime breakdown for a complete learner evaluation run on RTX 3060

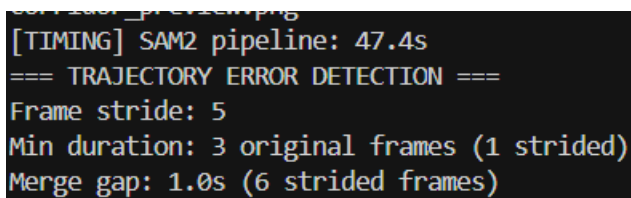
Pipeline stage	Runtime (s)
YOLO pre-pass (best.pt, all frames)	18.1
SAM2 blade tip tracking + trajectory	47.4
HSV angle estimation + DTW alignment	19.4
RAFT optical flow vibration detection	~7.0
MediaPipe hand distance estimation	~5.0
Trajectory error detection	< 0.1
Angle error detection	< 0.1
Error merge	< 0.1
Total	~97.0

The dominant cost is SAM2 tracking at 47.4 seconds, accounting for approximately 49% of total runtime. The YOLO pre-pass and HSV angle estimation each contribute roughly 18–20%, while RAFT and MediaPipe add under 8% each. All downstream evaluation and scoring stages complete in under one second combined, confirming that the computational bottleneck lies entirely within the perception layer. The improvement over the Phase 1 dual-video approach is clearly visible in the terminal output captured during a complete evaluation run. Figure 38 shows the YOLO pre-pass completing in 18.1 seconds, and Figure 39 shows the SAM2 tracking stage completing in 47.4 seconds — both representing learner-only processing with the expert reference already stored offline.



```
[TIMING] YOLO: 18.1s
[EVALUATE] Step 1/3 – YOLO detection complete
```

Figure 38 - Terminal output showing YOLO pre-pass timing



```
corridor_preempting
[TIMING] SAM2 pipeline: 47.4s
=== TRAJECTORY ERROR DETECTION ===
Frame stride: 5
Min duration: 3 original frames (1 strided)
Merge gap: 1.0s (6 strided frames)
```

Figure 39 - Terminal output showing SAM2 pipeline timing

These timings contrast directly with the Phase 1 benchmarking results in Table 16, where SAM2 alone required 60 seconds on the RTX 3060 for a 10-second video processed at one frame in three — a configuration that processed both expert and learner sequences together. The offline expert pre-processing architecture therefore not only reduces the per-evaluation runtime but also eliminates the need to reprocess the expert reference on every learner upload, making the system scalable to multiple concurrent learner evaluations against the same expert reference without additional computational cost.

5.5 Frontend Implementation

The AugMentor 2.0 frontend was implemented as a React Single Page Application built with Vite, communicating with the FastAPI backend exclusively through REST endpoints. The interface was designed around a dark theme with high visual contrast, chosen to keep the user's attention focused on the video content rather than the surrounding UI chrome. Navigation is organised across five main sections: Home, Courses, My Learning, Compare Studio, and History.

The Home interface, shown in Figure 40, serves as the entry point to the platform and introduces the system's core value proposition. A primary call-to-action directs users either to the course catalogue or directly to the Compare Studio, keeping the most common user journeys accessible without unnecessary navigation steps.

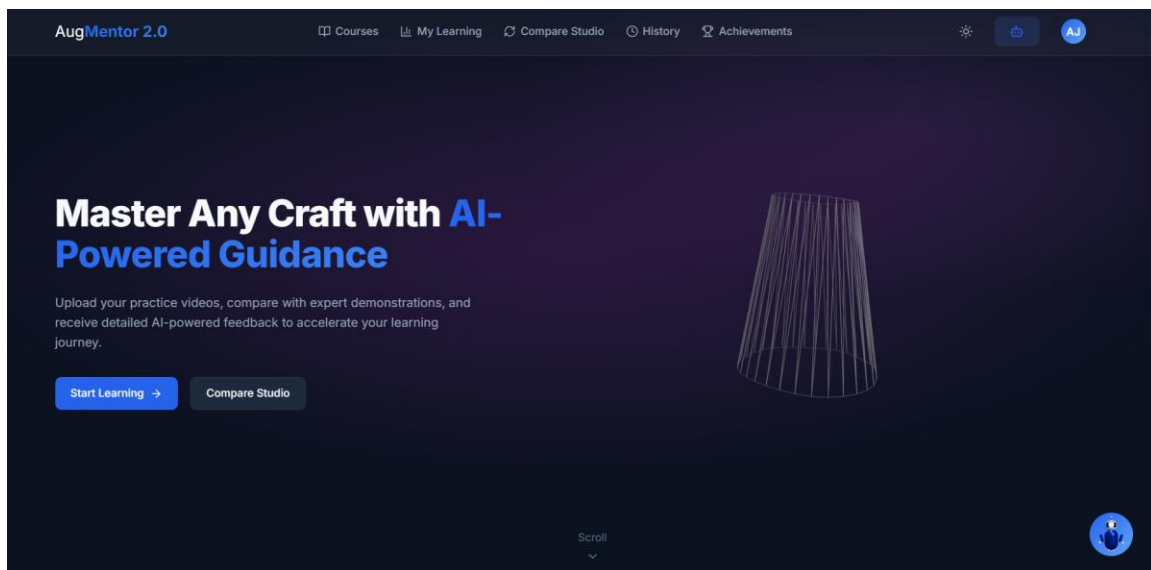


Figure 40 - Home interface

The Courses interface in Figure 41 organises available craft learning content by difficulty level — beginner, intermediate, and advanced — with each course card displaying the craft domain, estimated duration, and instructor information. Users navigate from the catalogue into individual course pages where expert demonstration videos are organised into structured chapters.

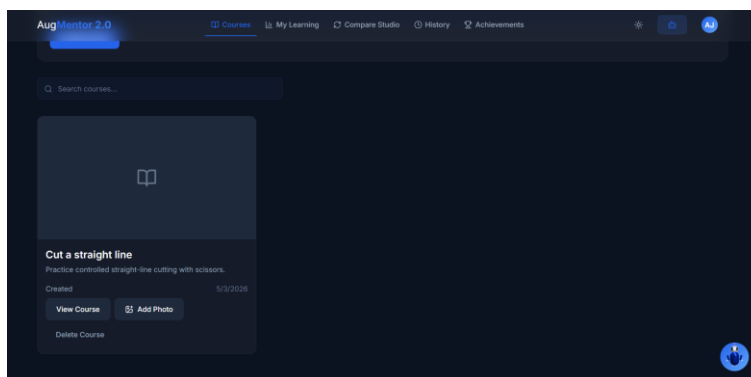


Figure 41 - Courses interface

Within each course, shown in Figure 42, learners can access the expert reference video for each chapter alongside their progress indicator. The chapter-based structure encourages incremental skill acquisition, with completion states tracked per chapter and persisted in the database across sessions.

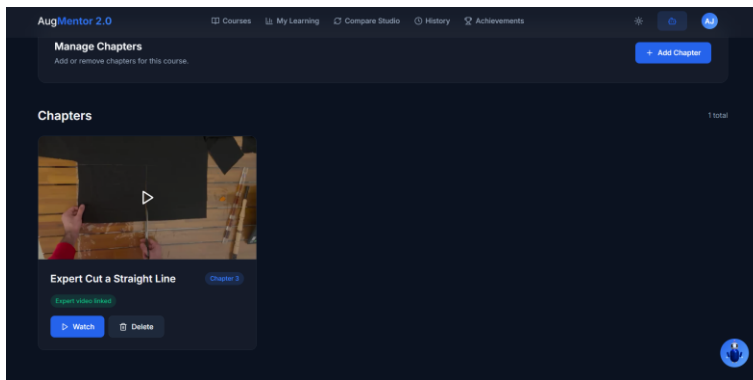


Figure 42 - Course content and chapters

The Compare Studio, shown in Figure 43, is the core interactive component of the platform. It presents the expert reference video and the learner's uploaded practice video in a synchronized side-by-side layout, with playback controls allowing the learner to adjust the timing offset between the two streams. The right panel exposes the evaluation tools tab — where the gamified error marking interface is accessible — and the feedback tab where the VLM-generated explanation is displayed once evaluation completes. Canvas-based overlays rendered on top of the learner video display the detected error annotations, corridor boundaries, and angle indicators at the relevant timestamps, synchronized to the HTML5 video playback position.

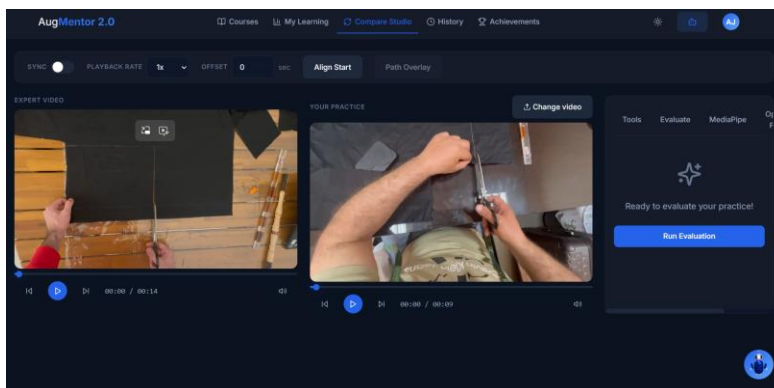


Figure 43 - Compare Studio

5.6 Implementation Challenges

Domain gap in tool detection

The most significant early challenge was the complete failure of general-purpose object detection models on the target domain. Off-the-shelf YOLOv8 variants trained on COCO produced irrelevant detections on every tested frame, making it impossible to establish the region of interest that all downstream pipeline stages depended on. This blocked all further development until resolved. The solution required building a domain-specific annotated dataset from scratch — extracting frames from the available egocentric videos, annotating bounding boxes manually, and fine-tuning YOLOv8s from ImageNet-pretrained weights. This was not anticipated in the initial design and consumed a significant portion of the Phase 1 timeline, but the resulting model achieved near-

perfect validation metrics and eliminated the detection problem entirely for the remainder of the project.

SAM2 GPU dependency and memory management

SAM2's memory-based mask propagation architecture places substantial demands on GPU VRAM. On the RTX 3060 with 6 GB VRAM, processing long learner videos required careful batch sizing and memory flushing between runs to prevent out-of-memory errors. CPU-only execution proved entirely impractical, exceeding 600 seconds for a 10-second video. The solution was to enforce GPU execution as a hard requirement, configure batch sizes conservatively, and implement explicit CUDA memory clearing between pipeline stages. Cross-hardware validation on the GTX 1650 confirmed that the pipeline remained functional on lower-specification hardware, albeit at significantly reduced throughput.

Corridor alignment across spatial reference frames

Aligning the expert corridor into the learner's spatial reference frame proved more complex than initially anticipated. The expert and learner videos were recorded from different egocentric viewpoints with different camera positions relative to the cutting surface, meaning the blade tip trajectories occupied entirely different regions of the image coordinate space. A naive overlay produced meaningless comparisons. The solution was an anchor-based translation alignment that matched the start point of the expert centreline to the start point of the learner's smoothed trajectory, computing a rigid translation vector and applying it to the full corridor geometry. Edge cases arose when the learner's starting position was ambiguous or when the trajectory cleaning stage removed too many early frames, requiring iterative refinement of the anchor selection logic and the segment rejection criteria.

VLM non-determinism and hallucination

Early testing of the VLM explanation module without structured constraints produced outputs that varied between runs, used internal pipeline terminology, and occasionally generated corrective instructions that contradicted the computed measurements. Controlling this behaviour required iterative prompt engineering — developing the forbidden terms list, enforcing the rigid output format, grounding each error description in the specific computed deviation values, and including both annotated learner frames and aligned expert frames as visual context. Setting temperature to 0.2 and seed to 42 reduced but did not eliminate phrasing variance, which was accepted as a residual limitation given that the core measurements and error identifications remained consistent across runs.

Asynchronous pipeline latency

With a total evaluation runtime of approximately 97 seconds, returning results synchronously from a REST endpoint was not feasible. Managing the asynchronous execution introduced its own challenges: the frontend needed to poll for completion, intermediate pipeline failures needed to be communicated clearly to the user, and partial results from completed stages needed to be preserved even when a later stage failed. FastAPI's BackgroundTasks mechanism handled the execution model adequately, but implementing robust error propagation and status reporting across the full pipeline required careful design of the evaluation run state machine and the database schema for storing intermediate results.

5.7 Test and Evaluation

5.7.1 End-to-End Evaluation Run

Error Detection Rules

The evaluation pipeline applies four independent error detection mechanisms, each governed by explicit mathematical rules. These rules are documented here in full to support reproducibility and to make the system's scoring behaviour transparent to both learners and supervisors.

Angle error detection operates exclusively on frames where the learner trajectory is inside the corridor. The angular difference between matched expert and learner frames is computed as the modulo-180 blade-angle distance, folded so values above 90° become 180° minus the difference, ensuring the metric always reflects the smaller of the two possible rotational directions. A frame is flagged when this difference strictly exceeds 15°. An error event is declared when 10 or more consecutive flagged frames accumulate. The event closes when the learner re-enters the corridor, or when a look-ahead of 90 inside-corridor frames finds 10 consecutive frames at or below the 15° threshold — in which case the accumulation is discarded as a self-correction. Error events separated by 0.5 seconds or less are merged into a single event.

Trajectory error detection samples the learner trajectory at a frame stride of 5 and computes the perpendicular signed-distance deviation from the nearest aligned centreline point at each sample, using the dot product of the displacement vector with the centreline normal. A sample is outside when this perpendicular deviation exceeds the corridor margin of 80 pixels. An error event requires at least 1 strided frame outside — equivalent to 3 original frames — and nearby events separated by 1.0 second or less are merged. Bounding boxes for spatial scoring are computed from all learner points in the error window with 80 pixels of padding.

Vibration error detection requires at least three consecutive one-second FFT windows with a confidence score above 0.15, a dominant frequency within ± 2 Hz across all three windows, and a locked frequency above 4 Hz to exclude the deliberate cutting rhythm. Severity is graded as mild for three windows with confidence below 0.20, moderate for three windows with confidence between 0.20 and 0.30, and severe for four or more windows or confidence above 0.30.

Gamified scoring matches learner-submitted marks to real errors using a spatial and type-based criterion. A mark is accepted as correct if its type label matches the error type and its pixel coordinate, scaled to the native 1920×1080 resolution, falls within the error bounding box. Each real error can be matched by at most one mark. The final score is:

$$\text{score \%} = \text{round}(100 \times \text{correct marks} / \text{total real errors})$$

False alarms are counted separately and do not subtract from the percentage.

Evaluation Setup

To validate the end-to-end pipeline under controlled and observable conditions, a deliberate stress test was designed. The expert video recorded a straight-line cutting motion — the simplest and most controlled reference trajectory possible. The learner video was recorded with intentionally exaggerated errors: a zigzag cutting path that deviated significantly from the expert's straight line, combined with sustained blade angle inconsistencies throughout the sequence. This setup was chosen not to represent a typical learner submission, but to verify that the pipeline reliably detects, localises, and explains clearly observable errors under worst-case deviation conditions. Successfully handling exaggerated errors is a necessary baseline before deployment to real learners whose mistakes would be subtler and less predictable.

The full pipeline completed successfully in approximately 97 seconds on the RTX 3060, processing the learner video from upload through YOLO pre-pass, SAM2 tracking, HSV angle estimation, DTW alignment, corridor evaluation, RAFT vibration detection, MediaPipe distance estimation, error detection, merging, gamified scoring, and VLM explanation generation without any stage failure or fallback.

Detected Errors

The pipeline detected four error events — two angle errors and two trajectory errors. Table 18 presents the full list with timestamps, durations, and peak deviation values.

Table 18 - Detected error events

Error ID	Type	Start (s)	End (s)	Duration (s)	Peak value
1	Angle	3.733	4.233	0.500	35.03°
2	Trajectory	4.333	8.167	3.833	226.07 px — right
3	Angle	8.267	10.233	1.967	60.76°
4	Trajectory	10.333	15.167	4.833	224.61 px — left

The sequencing of the errors reflects a behaviourally plausible pattern. The first angle error at 3.7 seconds preceded the first trajectory drift, indicating that the blade orientation diverged from the expert reference before the spatial path left the corridor — consistent with an incorrect cutting angle causing the blade to drift off course. The second angle error at 8.3 seconds reached a peak deviation of 60.76°, the largest recorded in the run, coinciding with the transition between the rightward and leftward trajectory excursions. The two trajectory errors reached peak deviations of 226 and 224 pixels respectively — nearly three times the corridor margin — confirming that the exaggerated zigzag motion was well outside the acceptable range and that both excursions were captured as distinct events with accurate directional labels.

No vibration event was detected in this run, correctly reflecting that the exaggerated zigzag motion, while spatially and angularly deviant, was performed with stable and controlled tool handling throughout. This confirms that the vibration detector correctly distinguished between deliberate large-scale spatial deviations and the high-frequency micro-oscillations that characterise involuntary hand tremor.

The overall DTW angle metrics are summarised in Table 19. A mean angle difference of 32.88° and a maximum of 80.61° confirm that the learner's blade orientation diverged substantially from the expert reference throughout large portions of the sequence, not only at the labelled error peaks.

Table 19 - DTW angle summary metrics

Metric	Value
Total DTW matches	862
Mean angle difference	32.88°
Median angle difference	34.43°
Max angle difference	80.61°
High error frames	346
Medium error frames	62
OK frames	192
Normalised DTW distance	23.02
Total errors detected	4
Trajectory errors	2
Angle errors	2

Gamified Scoring

The learner was presented with the annotated evaluation video and challenged to identify the errors through the gamified marking interface presented in Section 5.4.8. One correct mark was submitted, matching trajectory error 2 spatially and by type. Three errors were missed -- both angle errors and the second trajectory error — and no false alarms were submitted. The resulting score was 25%, reflecting one correctly identified error out of four real errors detected. The missed angle errors are consistent with the known difficulty of perceiving blade orientation deviations visually without a reference overlay, reinforcing the pedagogical value of the system's automated angle detection and VLM explanation for errors that learners cannot reliably self-identify.

VLM Explanation

The VLM explanation for this run addressed all four detected errors with specific timestamps, measured deviation values, and directional correction instructions. The structured output format — Overall, four Issue blocks each with a corrective instruction, Priority Fix, and closing positive observation — was preserved exactly as specified by the prompt architecture. No pipeline terminology appeared in the output and the feedback was phrased in natural language directly addressing the learner. The complete generated explanation is presented in Section 5.4.9. The result confirms that the metric-constrained prompt architecture successfully translates quantitative evaluation outputs into accessible, actionable pedagogical feedback even under the exaggerated deviation conditions of the stress test.

5.7.2 Vibration Detection Validation

The vibration detection module was validated independently from the main evaluation run using two purpose-recorded test videos. The expert video recorded a steady zigzag cutting pattern performed with controlled tool handling. The learner video recorded the same zigzag pattern with deliberate hand tremor introduced throughout. Both videos were processed independently through

the full RAFT optical flow pipeline and their per-window FFT results compared, as presented in Section 5.3.4.

The validation confirmed 100% detection accuracy across both cases — a correctly declared moderate severity vibration event on the tremor video, and a correct non-detection on the steady video — with zero false positives and zero false negatives.

The key discriminating signal was not the dominant frequency values themselves, which fluctuated across the full 2–14 Hz range in both videos, but the consistency of that frequency across consecutive windows. The tremor video produced three consecutive windows locked onto 14.48 Hz with a peak confidence of 0.236 at 11.0 seconds, satisfying all three detection conditions simultaneously. The steady video never produced more than one consecutive window at any frequency, yielding a consistency score of 0.026 compared to 0.086 in the tremor case. This threefold difference in consistency score provided a reliable discriminating signal that the rule-based classifier correctly acted upon.

The result demonstrates that the windowed FFT approach with consecutive-window locking is a more robust vibration discriminator than a raw confidence threshold alone. Isolated high-confidence windows — which appeared in the steady video at several timestamps — did not trigger a false positive because they were never sustained across three consecutive windows at a consistent frequency. This behaviour is by design: the classifier's requirement for temporal consistency specifically targets the sustained, repetitive nature of involuntary hand tremor rather than momentary motion spikes that can arise from any rapid tool movement.

Table 14 in Section 5.3.4 presents the full metric comparison between the two videos. The vibration module's integration into the unified error pipeline was confirmed by verifying that detected vibration events are passed correctly to the Error Merge Service as first-class error objects with bounding box, timestamp, severity, and peak frequency metadata, and that they appear correctly in the gamified marking interface and VLM prompt construction.

5.7.3 Critical Evaluation

Strengths

The pipeline's most significant strength is its deterministic, metric-driven architecture. Every error declaration results from explicit mathematical conditions applied to measured geometric quantities — perpendicular corridor deviation in pixels, modulo-180 angular difference in degrees, FFT confidence scores — with no black-box scoring or opaque weighting. This ensures that the system's outputs are reproducible, auditable, and directly explainable, satisfying the non-functional requirements for deterministic evaluation logic (NFR-A2) and grounded textual feedback (NFR-E1). The same input always produces the same error list and the same score — a property confirmed by repeated runs on identical video pairs. The VLM explanation layer introduces minimal residual variance in natural language phrasing between runs, but the underlying measurements, error identifications, and score are fully deterministic and unaffected by this variance.

The modular pipeline architecture proved its value throughout development. Each stage operates independently with well-defined inputs and outputs, allowing components to be developed, tested, and replaced without restructuring the broader system. The custom YOLOv8s model demonstrated the specific value of domain-specific fine-tuning: a general-purpose detector failed entirely on the task, while a fine-tuned model achieved near-perfect validation metrics and provided the stable region of interest that every downstream stage depended on. The offline expert pre-processing architecture further improved scalability by eliminating the need to reprocess the expert reference

on every learner upload, making evaluation latency independent of the number of registered experts.

The stress test validation confirmed that the system is robust under exaggerated deviation conditions. The pipeline correctly detected, localised, and explained four errors produced by an intentionally poor learner performance, with the VLM generating specific, actionable, and correctly directed feedback for each one. The correct non-detection of vibration during the zigzag run confirmed that the vibration classifier correctly distinguishes between large-scale spatial deviations and high-frequency tremor, avoiding false positives under challenging motion conditions.

Weaknesses and Limitations

The pipeline's primary operational limitation is its dependence on GPU acceleration. SAM2 accounts for 47.4 seconds of a 97-second total evaluation runtime and is entirely impractical without a CUDA-capable GPU. The total runtime, while acceptable for asynchronous background evaluation, cannot approach real-time without substantial architectural changes, limiting the system's applicability in contexts where immediate feedback is required.

The angle error detection rule that restricts flagging to inside-corridor frames is a deliberate design decision rather than a limitation — it avoids penalising the learner twice for what is essentially a single mistake, treating the angle deviation as the cause and the trajectory drift as the consequence. However, this means that in edge cases where a learner exits the corridor for a reason unrelated to blade angle — such as a sudden positional shift — a genuine concurrent angle error at that moment will not be captured.

The gamified scoring mechanism uses a spatial and type-based matching criterion with no temporal component. A learner who correctly identifies the location of an error but marks it at the wrong moment in the video receives full credit, while one who marks the correct time but slightly outside the bounding box receives none. A combined spatial and temporal matching criterion would produce a more discriminating and pedagogically meaningful score.

Finally, while the system supports multiple craft tasks and has been tested across different cutting scenarios, all validation was conducted using videos recorded under controlled conditions by the development team. Generalisation to real-world learner diversity — varying lighting conditions, different hand sizes, bare versus gloved hands, and users with no prior instruction — remains to be validated at scale and represents the primary direction for future work.

Table 20 - Requirements Satisfaction Summary

ID	Requirement Name	Priority	Status	Evidence / Reference
FR-1	User Authentication and Access Control	Must	Satisfied	Section 5.5 — login/register implemented in React SPA + FastAPI auth endpoints
FR-2	Course and Expert Content Access	Must	Satisfied	Section 5.5 — Courses interface, chapter navigation, expert video playback (Fig. 41–42)
FR-3	Practice Video Submission	Must	Satisfied	Section 5.5 — Compare Studio upload; format and duration validated before processing
FR-4	Domain Validation and Rejection	Must	Satisfied	Section 5.4.1 — YOLO pre-pass rejects videos with zero valid detections
FR-5	Structured Motion and Tool Extraction	Must	Satisfied	Sections 5.4.1–5.4.5 — YOLO, SAM2, HSV+fitLine, RAFT, MediaPipe all produce structured per-frame data
FR-6	Motion Alignment and Quantitative Evaluation	Must	Satisfied	Sections 5.4.6–5.4.7; Tables 18–19 — DTW alignment, corridor deviation, vibration FFT, distance metric
FR-7	Feedback Generation	Must	Satisfied	Section 5.4.9; Fig. 27 — annotated video output + VLM explanation referencing computed metrics
FR-8	Evaluation History Management	Should	Satisfied	Section 5.1 — PostgreSQL stores per-run metrics, scores, chapter references; History section in UI
FR-9	Gamification and Interactive Progress Tracking	Should	Satisfied	Section 5.4.8; Fig. 32–33 — gamified marking interface, score computation, chapter completion indicators
NFR-P1	Processing Latency	Performance	Satisfied	Table 17 — ~97s for 18-second video on RTX 3060 under async background task model
NFR-P2	Metric Reproducibility	Performance	Satisfied	Section 5.7.3 — deterministic pipeline; identical input produces identical error list and score across runs
NFR-R1	Fault Tolerance	Reliability	Satisfied	Section 5.4.1 — fallback logic for missing detections; invalid uploads rejected with informative response
NFR-R2	Input Format and Duration Constraints	Reliability	Satisfied	Section 5.4.1 — MP4/MOV enforced; 30s default cap validated at upload
NFR-S1	Secure Communication	Security	Satisfied	Section 3.2.1.3 — HTTPS/TLS mandated for all client–server communication
NFR-S2	Access-Controlled Storage	Security	Satisfied	Section 3.2.1.3 — user data accessible only via authenticated session
NFR-A1	Layered Architecture	Architecture	Satisfied	Section 4.2.1; Fig. 5 — six-layer component diagram with clear separation of perception, evaluation, scoring, and explanation
NFR-A2	Deterministic Evaluation Logic	Architecture	Satisfied	Section 5.7.1 — explicit mathematical rules for all four error types; score formula defined
NFR-E1	Grounded Textual Feedback	Explainability	Satisfied	Section 5.3.8; Fig. 27 — forbidden terms enforced; each issue block references computed metrics; Table 14 validates consistency

6 Impact of the Proposed Solution

Educational Impact

The most direct impact of AugMentor 2.0 is in vocational and craft education. Traditional apprenticeship models depend entirely on the physical presence of an expert who can observe, correct, and guide a learner in real time. This model is effective but inherently unscalable — a single expert can only supervise one learner at a time, feedback is subjective and inconsistent across sessions, and learners who practice unsupervised receive no correction at all. AugMentor 2.0 addresses this gap by providing structured, objective, and reproducible evaluation of egocentric craft video without requiring expert presence. A learner can upload a practice recording at any time and receive specific, timestamped feedback on blade angle, trajectory deviation, and tool vibration — error types that are genuinely difficult to self-identify without an external reference.

The gamified scoring mechanism introduces an additional pedagogical dimension: rather than passively receiving feedback, learners are first challenged to identify their own errors before the system reveals the full evaluation. This active engagement with the mistake identification process is more likely to produce durable skill improvement than passive feedback consumption, and the comparison between the learner's self-assessment and the system's objective evaluation gives both the learner and the supervisor a direct window into the learner's metacognitive awareness of their own technique.

Societal Impact

Artisan crafts represent a form of cultural and economic heritage that is increasingly at risk as traditional apprenticeship pathways decline and master craftspeople retire without successors. A scalable AI-assisted training platform lowers the barrier to entry for craft education by reducing the dependency on sustained one-to-one expert contact. Learners in geographically remote areas, or in institutions that cannot afford dedicated expert supervision for every student, gain access to structured evaluation that would otherwise be unavailable to them.

The collaboration between USJ and Mines Paris that frames this project reflects an international dimension: the expert video dataset originates from a professional craft context in France, while the system is developed and validated in Lebanon. This cross-border knowledge transfer — encoding expert craft knowledge into a reusable digital reference that can be evaluated against learners anywhere — is a concrete example of how AI-assisted education can bridge geographical gaps in access to expertise.

The system also has potential applicability beyond craft education. Any skill domain that involves repeatable hand-tool interactions captured on egocentric video — surgical training, musical instrument technique, dental procedure practice, industrial maintenance — shares the same structural problem that AugMentor 2.0 addresses: the need to compare a learner's fine motor execution against an expert reference and generate actionable feedback without requiring expert presence. The modular architecture of the pipeline makes extension to new domains a matter of retraining the YOLO detector and registering new expert references, rather than redesigning the system.

Economic Impact

From an institutional perspective, reducing the expert time required per learner evaluation has direct economic value. Expert craftspeople and trainers represent a scarce and expensive resource. A system that handles the routine evaluation workload — flagging specific errors, quantifying deviations, generating written feedback — allows expert time to be concentrated on the highest-value interactions: demonstrating advanced techniques, interpreting ambiguous cases,

and providing the kind of motivational and contextual guidance that no automated system can replicate.

For individual learners, access to objective feedback outside supervised sessions accelerates skill acquisition, potentially reducing the total training time and cost required to reach competency. In vocational training contexts where tuition fees and lost working time represent significant barriers, this efficiency gain has meaningful economic consequences.

Environmental Impact

A complete learner evaluation run requires approximately 97 seconds of GPU inference on an NVIDIA RTX 3060 Laptop GPU, which draws roughly 80–100 W under full load, corresponding to approximately 2–3 Wh of energy per evaluation. At the European average grid carbon intensity of approximately 250 gCO₂eq/kWh, a single evaluation produces well under 1 gCO₂eq — a negligible footprint compared to the primary alternative it displaces: physical travel by a craft expert to supervise a learner session in person. A 100 km round trip by car produces approximately 17 kg CO₂eq, a factor of more than 10,000 times the inference cost, meaning that as few as one or two displaced expert visits are sufficient to offset the embodied carbon of the GPU hardware itself over its service life. Cloud-hosted deployment would shift the inference load to data-centre hardware with potentially higher utilisation efficiency, but would introduce network transfer costs and raise data governance considerations under GDPR. As lighter segmentation architectures replace SAM2 in future iterations, the per-evaluation energy cost is expected to decrease further, strengthening the environmental case for AI-assisted vocational training over traditional expert-travel models.

Negative Impacts and Limitations

The system's dependence on GPU hardware introduces an access inequality: institutions or individuals without adequate computing resources cannot run the evaluation pipeline locally, and cloud deployment introduces cost and data privacy considerations. Learner videos capture the hands and working environment of real people, raising data governance questions that must be addressed before any production deployment, particularly under GDPR in European contexts.

There is also a risk of over-reliance on automated feedback at the expense of the human mentorship relationship. Craft learning involves tacit knowledge, aesthetic judgment, and cultural transmission that no metric-driven system can fully capture. AugMentor 2.0 is designed as a complementary tool, not a replacement for expert instruction, and this distinction must be clearly communicated to both institutions and learners to avoid the system being misused as a substitute for the human relationship at the core of apprenticeship.

7 Conclusion

AugMentor 2.0 set out to build an AI-powered platform capable of evaluating egocentric craft video against an expert reference and generating structured, explainable feedback for skill development. The delivered system achieves this objective across four error types — blade angle deviation, trajectory drift, tool vibration, and hand proximity — through a modular pipeline that combines a custom-trained object detector, a segmentation-based tracker, deterministic geometric analysis, signal processing, and a metric-constrained Vision–Language Model. The end-to-end pipeline runs in approximately 97 seconds per learner evaluation on consumer GPU hardware, produces fully reproducible error lists and scores, and generates natural-language feedback that is grounded in computed measurements and free of technical jargon.

The project's most significant technical contribution was the demonstration that fine-grained skill evaluation in egocentric craft video is achievable without large labelled training datasets or end-to-end deep learning evaluation models. The hybrid architecture — combining a small, domain-specific trained detector with deterministic geometric analysis and a constrained generative model — proved more practical, more interpretable, and more robust than any of the purely data-driven approaches evaluated during Phase 1. The failure of general-purpose object detectors, the prohibitive runtime of 3D hand reconstruction, and the non-determinism of unconstrained Vision–Language Models were all anticipated risks that the architecture successfully mitigated through targeted engineering decisions at each stage.

Positioned against the related work reviewed in Section 2, AugMentor 2.0 occupies a distinct space that no single prior system covers. Surgment demonstrates that fine-grained, region-level skill evaluation from video is achievable, but it operates within a controlled surgical domain with predefined instrument workflows and does not generalise to open craft settings where tools, viewpoints, and execution styles vary considerably; AugMentor 2.0 addresses the same evaluation objective requiring only a domain-specific fine-tuned detector — trained here on 584 annotated egocentric scissors images to achieve mAP@0.5 of 0.995 — and a single expert reference recording, rather than large labelled evaluation datasets, an approach that is directly replicable for new craft domains by retraining the YOLO component on a new tool dataset. V-JEPA 2.1 provided meaningful temporal localisation during Phase 1 benchmarking, correctly identifying the most divergent segment of the learner's performance with a similarity score of 0.458, but its embedding-based outputs cannot be mapped to measurable correction values — degrees of angular error or pixels of spatial drift — that a learner can act on; AugMentor 2.0 resolves this interpretability gap by replacing representation-level comparison with explicit threshold-based error detection, trading some holistic sensitivity for precision and reproducibility. Fast-HaMeR produced the richest geometric hand representation of any evaluated model, achieving an 80.8% detection rate under gloved egocentric conditions, but required approximately 200 seconds to process a 23-second video on the RTX 3060 — nearly nine times real-time — making it incompatible with online evaluation; AugMentor 2.0 achieves comparable error coverage through the combination of SAM2 tracking, HSV angle estimation, and RAFT optical flow at a total latency of approximately 97 seconds on the same hardware, remaining within the acceptable range for asynchronous background evaluation.

Honest reflection on the project's limitations is equally important. The system has been validated on a controlled stress test using a single craft task — straight-line cutting with scissors — performed under ideal recording conditions. Real-world deployment would expose the pipeline to learner diversity, lighting variability, different camera positions, and subtle errors that are far harder to detect than the exaggerated deviations used in validation. The gamified scoring evaluation produced a 25% score, reflecting the genuine difficulty of self-identifying fine motor errors without a reference — a result that validates the system's role but also highlights how much learner education about the error types themselves remains necessary alongside the automated feedback.

Several directions for future work emerge naturally from the current implementation. Extending the corridor alignment to support curved and multi-directional cutting patterns would significantly broaden the range of craft tasks the system can evaluate. Integrating a temporal component into the gamified scoring criterion would produce more discriminating and pedagogically meaningful scores. Reducing SAM2's runtime contribution through lighter segmentation architectures or frame-level approximation would bring the total evaluation latency closer to real-time. And validating the system with real learners across multiple craft domains — rather than the development team under controlled conditions — would provide the evidence base needed to assess the system's true pedagogical effectiveness.

AugMentor 2.0 demonstrates that the gap between expert craft knowledge and scalable digital evaluation is technically bridgeable with current AI tools, without sacrificing interpretability or reproducibility. The platform provides a concrete foundation for further research and development at the intersection of computer vision, educational technology, and artisan skill preservation.

8 References

- [1] Z. Chen, S. Ye, X. Chu, H. Xia, H. Zhang, H. Qu, and Y. Wu, "Augmenting sports videos with VisCommentator," *IEEE Transactions on Visualization and Computer Graphics*, vol. 28, no. 1, pp. 824–834, 2021.
- [2] X. Zhang, T. Dekel, T. Xue, A. Owens, Q. He, J. Wu, and W. T. Freeman, "Mosculp: Interactive visualization of shape and time," in *Proceedings of the 31st Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2018, pp. 275–285.
- [3] B. Lee, M. Cordeil, A. Prouzeau, B. Jenny, and T. Dwyer, "A design space for data visualisation transformations between 2D and 3D in mixed-reality environments," in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI)*, 2022.
- [4] K. Holstein, V. Aleven, and N. Rummel, "A conceptual framework for human–AI hybrid adaptivity in education," in *Artificial Intelligence in Education (AIED 2020)*, Springer, 2020, pp. 240–254.
- [5] F. Chollet, *Deep Learning with Python*. Manning Publications, 2018.
- [6] ISO/IEC 23053:2022, *Framework for Artificial Intelligence (AI) Systems Using Machine Learning*. ISO, Geneva, 2022.
- [7] ISO/IEC 27001, *Information Security Management Systems — Requirements*. ISO, Geneva, 2022.
- [8] ISO/IEC 25010, *Systems and Software Engineering — System and Software Quality Models*. ISO, Geneva, 2011.
- [9] Regulation (EU) 2016/679, *General Data Protection Regulation (GDPR)*. European Parliament, 2016.
- [10] Kinovea, Kinovea — Free and Open Source Video Analysis Software. Available: <https://www.kinovea.org/>
- [11] OpenAI, ChatGPT (GPT-5.5) [Large Language Model], 2026. Available: <https://openai.com/>
- [12] Cursor, Cursor — AI-Assisted Development Environment, 2026. Available: <https://cursor.sh/>
- [13] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978.
- [14] I. R. Dave et al., "FinePseudo: Improving Pseudo-Labeling through Temporal-Alignability for Semi-Supervised Fine-Grained Action Recognition," 2024.

- [15] C. S. Burlingham and D. J. Heeger, "Heading perception depends on time-varying evolution of optic flow," *Proceedings of the National Academy of Sciences (PNAS)*, 2020.
- [16] G. Pavlakos et al., "Reconstructing Hands in 3D with Transformers," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [17] N. Ravi et al., "SAM 2: Segment Anything in Images and Videos," *arXiv:2408.00714*, Meta AI, 2024.
- [18] M. Assran et al., "V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning," *Meta AI*, 2025.
- [19] L. Mur-Labadia et al., "V-JEPA 2.1: Unlocking Dense Features in Video Joint Embedding Predictive Architectures," 2026.
- [20] H. A. Jillani et al., "Fast-HaMeR: Boosting Hand Mesh Recovery Speed with RGB-D Inputs," 2025.
- [21] Z. Teed and J. Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," in *European Conference on Computer Vision (ECCV)*, 2020.
- [22] Object Management Group (OMG), "Unified Modeling Language (UML)," Version 2.5.1, formal, December 2017. Available: <https://www.omg.org/spec/UML/2.5.1/>

Appendix 1

Mathematical Modeling of the Evaluation Framework

This section formalizes the quantitative evaluation model used to compare a learner's motion with an expert reference. While perception modules (e.g., landmark detection and tool localization) rely on deep learning models, the evaluation itself is strictly deterministic and mathematically defined. This design ensures reproducibility, interpretability, and compliance with non-functional requirements related to determinism (NFR-A2) and metric stability (NFR-P2).

The goal of this model is to compute structured deviation measures that capture differences in posture, trajectory, dynamics, and interaction with tools.

Structured Motion Representation

A motion sequence is defined as a time-indexed set of landmark vectors:

$$X(t) = \{x_1(t), x_2(t), \dots, x_n(t)\}$$

Where:

$t \in [1, T]$: discrete time index (frame index)

$x_i(t)$: spatial position of landmark i at time t

n : number of tracked landmarks (e.g., 21 hand joints)

$x_i(t) \in \mathbb{R}^2$ or $\mathbb{R}^3$: coordinates in image or world space

Two sequences are considered:

$X_e(t)$: expert motion

$X_l(t)$: learner motion

This representation encodes the full temporal evolution of hand motion and forms the basis for all subsequent comparisons.

Spatial Normalization and Scale Invariance

To eliminate variations due to camera placement, user morphology, and scene scale, all landmarks are normalized:

$$\tilde{x}(t) = \frac{x(t) - \mu(t)}{s(t)}$$

Where:

$\tilde{x}(t)$: normalized landmark position

$x(t)$: original position

$\mu(t)$: reference origin (e.g., wrist center or centroid of landmarks)

$s(t)$: scale factor (e.g., distance between joints or bounding box size)

This transformation ensures that the evaluation is invariant to translation and scaling.

Interpretation:

The normalization step isolates **relative motion geometry**, ensuring that deviations reflect differences in execution rather than differences in viewpoint or body size.

Robustness to Noise and Missing Data

Real-world perception introduces noise and missing values. To mitigate this, three mechanisms are applied:

Temporal smoothing

$$x_{smooth}(t) = \frac{1}{k} \sum_{i=t-k}^t x(i)$$

Where:

k : temporal window size

$x(i)$: position at frame i

This reduces high-frequency noise and stabilizes trajectories.

Interpolation of missing values

$$x(t) = \lambda x(t - 1) + (1 - \lambda)x(t + 1)$$

Where:

$\lambda \in [0,1]$: interpolation coefficient

This reconstructs missing detections using temporal continuity.

Confidence-weighted error aggregation

$$D = \frac{1}{T} \sum_{t=1}^T w(t) \cdot error(t)$$

Where:

$w(t)$: confidence score for frame t

$error(t)$: instantaneous deviation

This reduces the impact of unreliable frames.

Interpretation:

These mechanisms ensure that the evaluation remains stable under imperfect detection conditions, while preserving deterministic behavior.

Temporal Alignment Using Dynamic Time Warping (DTW)

Since expert and learner sequences may differ in execution speed, temporal alignment is required.

$$d(i, j) = \| X_e(i) - X_l(j) \|$$

Where:

i, j : frame indices

$d(i, j)$: distance between expert frame i and learner frame j

The alignment cost is computed recursively:

$$DTW(i, j) = d(i, j) + \min \begin{cases} DTW(i-1, j) \\ DTW(i, j-1) \\ DTW(i-1, j-1) \end{cases}$$

Final cost:

$$D_{DTW} = DTW(T_e, T_l)$$

Where:

T_e, T_l : sequence lengths

Interpretation:

DTW finds an optimal mapping between sequences, allowing comparison of motions that are temporally misaligned but structurally similar.

Joint Angle Deviation

Joint angles are computed from three landmarks:

$$\theta = \cos^{-1} \left(\frac{(A - B) \cdot (C - B)}{\|A - B\| \|C - B\|} \right)$$

Where:

A, B, C : 3D landmark positions

B : joint center

θ : angle at joint

Deviation is computed as:

$$D_{angle} = \frac{1}{T} \sum_{t=1}^T |\theta_e(t) - \theta_l(t)|$$

Where:

$\theta_e(t), \theta_l(t)$: expert and learner angles

Interpretation:

This metric captures differences in articulation and posture, independent of global position.

Trajectory Deviation

$$D_{traj} = \frac{1}{T} \sum_{t=1}^T \|x_e(t) - x_l(t)\|$$

Where:

$x_e(t)$, $x_l(t)$: corresponding landmark positions

Interpretation:

This measures spatial deviation between motion paths and captures whether the learner follows the same trajectory as the expert.

Velocity Profile Difference

$$v(t) = \frac{x(t) - x(t-1)}{\Delta t}$$

Where:

$v(t)$: instantaneous velocity

Δt : time step between frames

$$D_{vel} = \frac{1}{T} \sum_{t=1}^T \|v_e(t) - v_l(t)\|$$

Interpretation:

This metric captures temporal dynamics, including speed, rhythm, and motion smoothness.

Tool Alignment Deviation

$$D_{tool} = \frac{1}{T} \sum_{t=1}^T \|c_e(t) - c_l(t)\|$$

Where:

$c_e(t)$, $c_l(t)$: tool positions

Interpretation:

This evaluates how accurately the learner interacts with tools relative to the expert.

Optical Flow-Based Motion Consistency

$$D_{flow} = \frac{1}{T} \sum_{t=1}^T \| F_e(t) - F_l(t) \|$$

Where:

$F_e(t), F_l(t)$: optical flow fields

Interpretation:

This metric captures fine-grained motion patterns that may not be represented by discrete landmarks, such as micro-movements and motion continuity.

Representation-Space Similarity (Embedding-Based)

$$z_e(t) = E(X_e(t)), z_l(t) = E(X_l(t))$$

$$D_{embed} = \frac{1}{T} \sum_{t=1}^T \| z_e(t) - z_l(t) \|$$

Where:

$E(\cdot)$: encoder function

$z_e(t), z_l(t)$: latent representations

Interpretation:

This metric captures high-level semantic similarity between motion sequences, complementing geometric measures.

Aggregated Deviation Model

$$D_{total} = \sum_{i=1}^6 w_i D_i$$

Where:

D_i : individual deviation metrics

w_i : weights such that $\sum w_i = 1$, $w_i \geq 0$

Interpretation:

This combines multiple aspects of motion into a single scalar value while preserving interpretability.

Uncertainty and Confidence Modeling

$$D_{total}^{adj} = D_{total} \cdot (1 - \sigma)$$

Where:

σ : uncertainty measure

Interpretation:

Higher uncertainty reduces confidence in the final score, ensuring robustness under unreliable conditions.

Performance Score Normalization

$$Score = 100 \cdot e^{-\alpha D_{total}}$$

or

$$Score = 100 - \beta D_{total}$$

Where:

α, β : scaling parameters

Interpretation:

These functions map deviation to a bounded score while preserving monotonicity.

Deterministic Grounded Feedback Integration

$$Input_{VLM} = \{D_{angle}, D_{traj}, D_{vel}, D_{tool}, D_{flow}, D_{embed}, Score\}$$

The Vision–Language Model operates only as an interpretation layer and does not modify computed results.

Interpretation:

This ensures that explanations remain grounded in quantitative evidence, aligning with explainability (NFR-E1) and deterministic evaluation principles.

Appendix 2 - Declaration of AI Usage

Student Name (s) : Ali Azzam, Ahmad Dia, Rafic Dergham

Project Title: AugMentor 2.0: Vision–Language–Action Authoring for Egocentric Craft Video Annotation

AI Usage Disclosure

Please check **all** statements that apply to this project:

☐ **No AI used**

The work is entirely my own and was completed without the assistance of generative AI tools.

☒ **Grammar & Style Assistance**

AI tools were used only to improve grammar, spelling, clarity, or writing style of text originally written by me.

☒ **Technical Support**

AI tools were used for non-substantive technical assistance such as code debugging, data visualization suggestions, or brainstorming ideas.

☐ **Content Generation**

AI tools were used to draft or generate specific sections of the project (requires detailed explanation below).

Detailed Description of AI Usage

(Required if any option other than “No AI used” is selected)

Claude (Anthropic, Claude Sonnet 4.6) was used to refine and improve the clarity, structure, and academic tone of sections written by the team, including the technical documentation in this report, the impact analysis, the conclusion, the abstract, and the résumé. All information was originally produced by the team and subsequently refined for grammar, consistency, and readability. All technical values, results, error detection rules, and system behaviour described in the report were verified directly from the codebase and evaluation outputs by the team.

ChatGPT (OpenAI, GPT-4o) and Cursor were used for editorial assistance and development support during implementation. GitHub Copilot was used for code suggestions during backend development, with all suggestions reviewed and validated before integration.

9 Certification

I certify that:

- All AI-assisted content has been critically reviewed for accuracy and correctness.
- I fully understand and can explain all parts of this project.
- The final submission reflects my own understanding and effort and complies with the institution's academic integrity policies.

Student Signature:

Ali Azzam: Date: 15/05/2026

Ahmad Dia:Date: 15/05/2026

Rafic Dergham:Date: 15/05/2026